IRI.MAPTOR · MAKANNEGAR SABA

Spatial Versioning

Feature-level pending changes, competitive review and a fixed promotion pipeline

DOCUMENT	Design dossier — consolidated stages 1–6
STATUS	**Baselined** — 2026-08-13 · decisions D1–D44 settled, no open questions
IMPLEMENTATION	Started 2026-08-13 at milestone M1 (Foundation)
PILOT SCOPE	Transmission lines + substations (D38)
SOURCE	docs/features/spatial-versioning/

CONTENTS

Generated from the six stage documents — the markdown files remain the source of truth.

Contents

03 Data Model

1. Design principles
2. Entity catalog
3. Relationships (summary)
4. Enums (shared core)
5. Key mechanics
6. EF Core & project integration
7. Sizing & performance notes
8. Stage-3 detail questions (Q30–Q36) — all ANSWERED 2026-08-13
9. Handoff to Stage 4 (architecture & integration)

04 Architecture & Integration

1. Component overview
2. Project layout and boundaries [A: D29]
3. API surface (v1 sketch)
4. Gating the direct Sync path [A: D26]
5. Client state after submission [A→D34, Q37 modified by Hossein]
6. Sequence flows (component level; semantics in doc 02)
7. Authorization mapping [A: FR-4.3, doc 02 §6]
8. Cross-cutting
9. Stage-4 detail questions (Q37–Q40) — all ANSWERED 2026-08-13
10. Handoff to Stage 5 (UI/UX)

05 UI/UX Specification

0. Principles
1. Navigation & map entry points
2. Submit flow
3. My Pending panel (dockable)
4. Review Queue (VersionReview)
5. Competition Compare view (the centerpiece)
6. Approval Queue (VersionApprove)
7. Inbox
8. History timeline (per feature)
9. Color & badge semantics (theme tokens at implementation)
10. Failure & empty states
11. Stage-5 detail questions (Q41–Q44) — all ANSWERED 2026-08-13
12. Handoff to Stage 6 (implementation plan)

06 Implementation Plan

1. Ground rules
2. Milestones
3. Test strategy
4. Risk register (implementation-phase)
5. Rollout & rollback
6. Definition of done (planning → implementation)
7. Stage-6 operational questions (Q45–Q46) — ANSWERED 2026-08-13

STAGE 1 OF 6

Requirements & Boundaries

`docs/features/spatial-versioning/01-requirements-and-boundaries.md`

> **Status:** DRAFT — under discussion with Hossein **Last updated:** 2026-08-13 **Source prompt:** `docs/features/spatial versioning prompt design.txt`

0. How to resume this work (read this first)

This is a multi-day planning effort. The plan is a sequence of stage documents, all in `docs/features/spatial-versioning/`:

Stage	Document	State
1	`01-requirements-and-boundaries.md` (this file)	Baselined 2026-08-13 (D1–D18; Q15/Q17 still open)
2	`02-workflow-and-state-machine.md` — lifecycle, states, transitions, role matrix	Baselined 2026-08-13 (Q23–Q29 confirmed → D19–D25)
3	`03-data-model.md` — EF Core entities, history strategy, concurrency	Baselined 2026-08-13 (Q30–Q36 → D27–D33; Q34 modified)
4	`04-architecture-and-integration.md` — which libraries, Saba API surface, boundaries	Baselined 2026-08-13 (Q37/Q40 modified → D34/D37; Q38/Q39 → D35/D36)
5	`05-ui-ux.md` — WPF views: session editor, competition compare, review queue	Baselined 2026-08-13 (Q41–Q44 → D39–D42)
6	`06-implementation-plan.md` — milestones, pilot scope	Baselined 2026-08-13 (Q45–Q46 → D43–D44) — **planning complete; M1 started 2026-08-13**

To resume: read §2 (Decisions so far), then §9 (Open questions) and pick the next **Open** item. Do not start a later stage while its blocking questions are Open. Every settled decision gets a row in §2 with a date — nothing is decided in chat only.

1. Problem statement

Editors of a spatial database (features = geometry + attributes) must be able to propose changes to individual features without touching the live state. Proposed changes are grouped into **version sessions**, submitted for review, and flow through a fixed moderation pipeline. Uniquely, the process is **deliberately competitive**: multiple editors may be assigned the same feature/area and submit competing proposals; a reviewer compares them side by side (geometry diff + attribute diff), selects one winner, and rejects the rest with reasons. Selected changes are then committed to the live state. Full decision history is preserved.

This is explicitly **not** ESRI layer-versioning: granularity is the individual feature, and there is no long-lived branch of a whole layer.

2. Decisions so far

#	Date	Decision	Rationale / consequence
D1	2026-08-13	Competition is a **deliberate business process**, not just accidental-conflict resolution.	Competition (and probably work assignment) becomes a first-class concept. Accidental collisions still occur and must funnel into the same compare-and-select mechanism.
D2	2026-08-13	**Central DB, always online.** All editors reach one SQL Server through the Saba API.	Pending changes live server-side from submission. Offline/check-out editing is out of scope.
D3	2026-08-13	**Saba is the first consumer; the core is reusable.** Versioning core (change model, diff, workflow state machine) goes into shared IRI.Maptor libraries; Saba-specific glue stays in Saba.	Core must not depend on Saba entity types.
D4	2026-08-13	Pipeline is **fixed, not configurable**. Number of stages still undecided (→ Q5).	No workflow-configuration engine. One person may hold multiple roles (small-team collapse) unless Q20 decides otherwise.
D5	2026-08-13	Pipeline = **3 fixed stages**: Edit → Review → Approve & Commit.	Approve is the final gate (stale re-check, transactional batch commit). Role overlap per D10. Resolves Q5.
D6	2026-08-13	**Session = unit of submission only; decisions are per-feature.** Sessions may end partially accepted; session state is derived, not authoritative.	Resolves R2/Q6 as recommended.
D7	2026-08-13	**Blind competition.** Editors never see rival pending proposals; only reviewers/approvers see competitors.	Visibility detail in D18. Post-closure visibility still open (→ Q25 in doc 02).
D8	2026-08-13	Stale proposals: **warn + explicit recorded override** at acceptance; staleness is re-checked (and overridable) again at approval/commit.	Resolves R3/Q14.
D9	2026-08-13	Session lifetime is **flexible**: multiple drafts per editor; withdraw while nothing decided; no auto-expiry; proposal age shown prominently in the review queue.	Resolves Q13; partially mitigates R9.
D10	2026-08-13	The same person **may** review and approve the same competition; both decisions are identity-stamped.	Separation of duty = organizational policy, not a system constraint. Resolves Q20.
D11	2026-08-13	Sessions are **single-editor**.	Resolves Q19.
D12	2026-08-13	Delete proposals are **ordinary competitors**; the compare UI renders deletion as a first-class side.	Resolves Q9/R7.
D13	2026-08-13	History (rejected + committed) is kept **forever**; no purge in v1.	Resolves Q11.
D14	2026-08-13	Draft undo/redo stays the existing **in-memory** mechanism; nothing persisted.	Resolves Q12.
D15	2026-08-13	v1 notifications = **in-app inbox, polling**.	Resolves Q16.

#	Date	Decision	Rationale / consequence
D16	2026-08-13	**No assignment mechanics.** Editors edit whatever they have access to; competitions form **implicitly by feature-id collision**. Spatial overlap is advisory only: warnings to editors against **live** features; overlap **suggestions** (incl. pending-vs-pending) go to reviewers, who may **manually group** proposals (esp. competing creates) into one competition.	"Deliberate" competition is orchestrated outside the system. Consequences analyzed in doc 02 §1. Replaces the assignment half of S3; resolves Q8/Q10.
D17	2026-08-13	Approver **return reopens** the competition: losers' rejections stay provisional until commit; final rejection notifications fire only at commit; on return all proposals revert to Submitted and the reviewer sees the return reason.	Resolves Q21.
D18	2026-08-13	Blind detail: a competing editor sees **status + competitor count** on their own proposal — never rival content or author names.	Resolves Q22. *(Author-hiding later amended by D34.)*
D19	2026-08-13	Late arrivals during approval open a **queued competition**: at most one Resolved + one Open competition per feature; the queued one cannot resolve until its predecessor is terminal.	Resolves Q23. Stage-3 constraint: filtered unique indexes per competition state.
D20	2026-08-13	Reviewer may close a competition **with no winner, without approver involvement** — rejections are final immediately (live is untouched).	Resolves Q24. Fully recorded in decision history.
D21	2026-08-13	Post-closure visibility: **participants** see the full record of competitions they took part in (rival content + authors); reviewers/approvers see all; non-participants see committed history only.	Resolves Q25. Widening later is a policy switch.
D22	2026-08-13	**Self-supersede**: one active proposal per (editor, feature); a newer submission auto-withdraws the editor's older pending proposal on that feature.	Resolves Q26. Stage-3 filtered unique index.
D23	2026-08-13	**Orphaned proposals** (target deleted in live) are reject-only in v1; no accept-as-recreate.	Resolves Q27.
D24	2026-08-13	Notifications aggregate as **per-session digests** — never one inbox row per feature.	Resolves Q28. Required by R5 throughput.
D25	2026-08-13	**Per-proposal withdraw** allowed while the proposal's competition is Open; a selected winner cannot withdraw during approval.	Resolves Q29.
D26	2026-08-13	**Per-layer Sync gate**: layers with versioning enabled reject the direct `Sync` write path with a clear error; non-versioned layers keep today's behavior.	Resolves Q17/S6. Every live write on a versioned layer provably went through review; copy-on-write history stays complete.
D27	2026-08-13	Draft sessions are **client-side only**; the server session row is created at submission.	Resolves Q30. A crash loses only the local draft — same as today.
D28	2026-08-13	History = **copy-on-write at commit**; no baseline snapshot; a layer's timeline starts at its enablement.	Resolves Q31.
D29	2026-08-13	Packaging: new shared projects `IRI.Maptor.Sta.Versioning` (EF-free model) + `IRI.Maptor.Ket.VersioningPersistence` (EF + services); all tables in SQL schema `versioning`.	Resolves Q32; implements D3.

#	Date	Decision	Rationale / consequence
D30	2026-08-13	Overlap suggestions are **persisted** as dismissable rows, computed once at submission.	Resolves Q33.
D31	2026-08-13	User references = scalar **UserId + display name stored explicitly at write time**; no FK to Saba Users; names are NOT resolved at query time.	Q34 accepted **in modified form** (Hossein). Matches the existing `FeatureBaseEntity.CreatedByFullName` convention; history survives any account lifecycle.
D32	2026-08-13	Proposal/history geometry columns are typed **SQL** `geometry` with spatial indexes.	Resolves Q35. Enables the overlap scan.
D33	2026-08-13	Schema signatures are **auto-computed from the EF model at API startup** and stamped into the layer registry.	Resolves Q36. Drift is detected, never declared.
D34	2026-08-13	**Live-truth display + strict query separation** (Q37 modified by Hossein): the map always renders the last committed state; versioning data never rides on normal feature/list endpoints. Two on-demand entry points only: (1) a layer query returning the editor's **own** pending proposals as separate overlay features; (2) a per-feature **pending-status check** returning the **count + authors** of pending proposals (usable while editing, even for others' proposals). No automatic versioning calls inside the normal edit flow.	**Amends D18**: rival *authors* become discoverable on demand; rival *content* stays hidden until closure (D21) — the anti-anchoring core of blind competition survives.
D35	2026-08-13	Dedicated `/versioning/*` **controllers** — one bounded context, no changes to domain controllers.	Resolves Q38.
D36	2026-08-13	Client gateway (`VersioningWebClient` + `VersioningWebDataSource`) lives in `IRI.Maptor.Ket.WebApiPersistence`, beside `WebApiDataSource`.	Resolves Q39; serves the D3 reuse goal.
D37	2026-08-13	**No polling in v1** (Q40 modified by Hossein): inbox, own-proposal statuses, and queues refresh manually / on open only. May be improved later.	A timer or push can be added later without API changes.
D38	2026-08-13	**Pilot layers: transmission lines + substations.**	Resolves Q15. Exercises vertex-heavy line edits, point/polygon edits, and all change types.
D39	2026-08-13	Compare view uses a single **N-way attribute grid** (live + one column per proposal).	Resolves Q41.
D40	2026-08-13	Bulk-accepting singleton proposals takes **one confirmation** (count + expandable list); per-row failures reported afterward.	Resolves Q42; serves R5 throughput.
D41	2026-08-13	History versions render as an **overlay** beside current, in a distinct style — never replacing live rendering.	Resolves Q43; consistent with D34's live-truth principle.
D42	2026-08-13	Versioning views live **inside** `IRI.Maptor.Jab.Wpf` (own folder, beside `FeatureChangesView`); no separate UI project for v1.	Resolves Q44.
D43	2026-08-13	Disabling versioning on a layer is **blocked while open proposals exist** — the admin resolves or withdraws them first.	Resolves Q45. No frozen-proposal state; history never gains a gap.

#	Date	Decision	Rationale / consequence
D44	2026-08-13	**The current working system serves as staging** (Hossein: data loss there is acceptable); M1's migration test and M6's dry run happen on it directly.	Resolves Q46.

3. Glossary

Term	Meaning
Feature	One spatial record: geometry + attribute set, living in one live table. In Saba: a row of a `FeatureBaseEntity` descendant.
Live state	The current committed row in the real feature table — what every viewer sees on the map.
Pending change	One proposed create / update / delete of one feature, not yet applied to live. Carries the full proposed state, its author, and the live version it was based on.
Version session	A named batch of pending changes by one editor, with timestamp and optional comment. Unit of *submission*, not necessarily of *review* (→ Q6).
Competition	The set of pending changes targeting the same feature (or the same assignment). Resolved by selecting exactly one winner and rejecting the rest.
Promotion	Moving a pending change forward one pipeline stage (e.g., selected by reviewer → awaiting approval).
Commit	Applying an approved change to the live table. The only write path to live data under this system.
Rejection	A recorded decision against a pending change, with a mandatory reason; the change is archived, never physically deleted.

4. What we need — requirements

Marked **[A]** = agreed (follows from D1–D4 or the prompt), **[P]** = proposed by Claude, needs Hossein's confirmation.

FR-1 Pending changes at feature granularity

1. **[A]** An edit produces a pending change against one feature; live state is untouched until commit.
2. **[A]** Change types: create, update (geometry and/or attributes), delete.
3. **[P]** A pending change stores the **full proposed state** (geometry + all attributes), not a delta. Deltas are computed for display only. (Robust against base drift; trivially applies on commit; costs storage — see S2.)
4. **[A]** Each pending change records the live version it was based on (base `RowVersion`), so staleness is detectable.

FR-2 Version sessions

1. **[A]** A session groups pending changes by **one** editor (multi-editor sessions rejected — see R2/Q19) with created/submitted timestamps and an optional comment.
2. **[A]** Sessions support bulk content (hundreds of features).

3. **[A]** Lifecycle at minimum: draft (editable, private) → submitted (visible to reviewers) → fully resolved (every contained change accepted/rejected). Exact states in Stage 2.
4. **[A]** (D9) Sessions can be withdrawn by their editor while no contained change has been decided; multiple draft sessions per editor are allowed; no auto-expiry.

FR-3 Competition & review

1. **[A]** Multiple pending changes may target the same feature simultaneously; the system never blocks this with locks.
2. **[A]** Reviewers can discover, per feature, all competing pending changes.
3. **[A]** Side-by-side comparison: geometry diff (visual, on map) + attribute diff (field-level table).
4. **[A]** Reviewer accepts exactly **one** competitor; all others in that competition are rejected. N-way, not just binary.
5. **[A]** Every rejection carries a mandatory reason; the author is notified.
6. **[P]** Non-competing changes (single proposal for a feature — expected to be the majority) go through a fast path: bulk review/accept at session level, not feature-by-feature. Without this, reviewers drown (see R5).
7. **[P]** Merging non-overlapping competitors (one edits geometry, other edits attributes) is **deferred** — not in stage 1 (see Q18).

FR-4 Fixed promotion pipeline & roles

1. **[A]** (D5) Fixed 3-stage pipeline: Editor → Senior Reviewer → Approver → live.
2. **[A]** Roles: Editor, Senior Reviewer, Approver, Viewer.
3. **[A]** Roles map onto Saba's existing `Role` / `Permission`-enum authorization model — no parallel permission system.
4. **[A]** Detailed action/visibility matrix is a Stage 2 deliverable.

FR-5 History & auditability

1. **[A]** Every decision (accept / reject / approve / commit) is recorded: who, when, why.
2. **[A]** Rejected changes are archived, never deleted.
3. **[A]** Committed history allows answering "what did this feature look like before commit X, and who changed it".
4. **[A]** (D13) Retention is indefinite; revisit only if volume forces it.

FR-6 Notifications

1. **[A]** Editors are notified when their change is rejected (with reason) or committed.
2. **[A]** (D15) Stage 1 delivery = in-app inbox (polling). No push infrastructure exists in the WPF client today; building one is out of scope (see §6).

NFR (non-functional)

1. **[A]** No layer-level locking, ever. Concurrency is optimistic; divergence is resolved by review, not prevented.
2. **[A]** Bulk sessions of ~500 features must be practical to submit and to review (drives FR-3.6).
3. **[P]** Commit must be transactional per accepted group: an approved batch either fully applies or not at all.

4. **[P]** The versioning core must not reference Saba assemblies (D3); it operates on a generic feature representation (table identifier + serialized geometry/attributes).

---

5. Boundaries

In scope (this feature, first delivery)

- Saba (MakanNegar Saba) as first consumer; central SQL Server behind the existing API (D2, D3).
- Feature-level pending changes, sessions, competitive review, fixed pipeline, decision history, in-app notifications.
- WPF UI in the existing client stack (which library hosts it → Stage 4/5).

Out of scope (explicitly rejected)

- ESRI-style layer versioning / long-lived layer branches.
- Offline editing, check-out/check-in, replica sync (D2).
- Configurable workflow engine (D4).
- Automatic merge of competing changes (deferred, Q18).
- Multi-step persisted undo/redo for editors (in-session undo stays the existing in-memory mechanism; D14).
- Real-time push notifications (SignalR/gRPC streaming) — stage 1 is polling (D15).
- Assignment/task management (D16) — competitions form by feature-id collision only; work orchestration stays outside the system.

Deferred (worth doing later, not now)

- Topology-aware validation of accepted sets (shared edges, connectivity) — stage 1 gives advisory warnings only (R4).
- Attribute/geometry merge of non-overlapping competitors (Q18).
- Extending versioning to other apps (AlborzNegar, NiocExp, …) — design for it (D3), don't build it.

---

6. Current state of the codebase (constraints we must design around)

Found 2026-08-13 by code exploration; verify before relying on details.

- **Client edit flow today:** `MapViewer` / `MapViewModelBase.EditAsync` → `SelectedLayer` commands → `EditableFeatureLayer` (geometry) / `FeatureTable` (attributes) → `BaseLayer.SaveChangesAsync()` → `IEditableVectorDataSource.SaveChangesAsync()`.
- **Feature model:** `Feature<T>` (`IRI.Maptor.Sta.Spatial/Primitives/FeatureSets/FeatureOfT.cs`) already has `Status` (`FeatureStatus`), a single in-memory `OldVersion` snapshot, `Guid Key`, and `AreTheSame` comparison. All change tracking is in-memory and discarded on save — nothing is persisted.
- **Wire unit of work:** `FeatureSetChangesDto` (Added/Updated/Deleted) — conceptually very close to a "version session"; the natural seam for capturing sessions.

- **Saba write path:** `WebApiDataSource.SaveChangesAsync` → `PUT /<Domain>/Sync<Entity>` → MediatR → `UnitOfWork.SyncFeatureEntitiesAsync` → `BargContext.SaveChangesAsync`, with optimistic concurrency via `RowVersion` round-tripped inside the attribute dictionary. **This direct-to-live path must be closed (or gated) for versioned layers** — otherwise versioning is advisory only (→ Q17).
- **Saba entity model:** ~100 concrete feature tables inherit `FeatureBaseEntity` (`Id`, `SHAPE`, `ObjectId`, `gis_id`, audit stamps, `RowVersion`). A pending-change store must reference features **polymorphically** (table/entity identifier + id) — one FK per table is impossible (R1).
- **Spatial stack:** custom Maptor EF Core provider (no NetTopologySuite), geometry as SQL Server native binary/WKB. Any history/diff storage must use the same representation.
- **No temporal tables** anywhere today; the existing hash-chained `AuditLog` covers *security* events only, not feature edits.
- **Diff UI seed:** `FeatureChangesViewModel` + `FeatureChangesView` already do attribute diff and geometry compare for unsaved edits — a starting point for the competition compare view.
- **Roles:** `User` / `Role` / `UserRole` / `RolePermission` with a flat `Permission` enum, one API policy per value. New versioning permissions = new enum entries in a new group.

---

7. Critique of the proposed model

Strengths

- Feature-level granularity fits reality: edits are sparse; ESRI layer versioning would version mostly-unchanged data and impose heavyweight reconcile/post semantics.
- Sessions match both the editor's mental model ("today's work package") and the existing `FeatureSetChangesDto` seam — low-friction capture.
- Deliberate competition is an unusual but coherent QA mechanism, and it subsumes accidental conflicts: a collision is just an unplanned competition.
- Storing pending changes server-side (D2) avoids the entire sync/replica problem space.

Weaknesses & risks (numbered; referenced elsewhere)

- **R1 — Polymorphic target + serialized state.** With ~100 feature tables, pending changes must store proposed state generically (geometry blob + attribute JSON + table identifier). This is unavoidable, but it creates a **schema-drift hazard**: an EF migration adds/renames a column while pending changes exist → their serialized attributes no longer match the schema. Mitigation: version-tolerant apply + validation gate at accept/commit time; a migration checklist item. This is the single biggest hidden cost of the design.
- **R2 — Session is the wrong unit of review.** THE central tension. If review decisions are per-feature (they must be, for competition), a 500-feature session will be *partially* accepted — some features win, some lose, some still pending. So a session cannot be atomic, and "session state" is derived, not authoritative. Recommendation: session = unit of **submission and provenance** only; competition = unit of **decision**; approved batch = unit of **commit**. If Hossein wants all-or-nothing sessions instead, the competition mechanic breaks — the two are mutually exclusive (→ Q6). *(Accepted 2026-08-13 → D6.)*

- **R3 — Base drift / lost updates via review.** A proposal is made against live version X. Before it's reviewed, a competing change gets committed → live is now X+1. Accepting the old proposal silently overwrites X+1's content. Since full proposed state is stored (FR-1.3), the overwrite is total, not partial. Mitigation: every pending change carries its base `RowVersion`; stale changes are flagged at review, and accepting a stale change requires an explicit override (→ Q14). *(Accepted 2026-08-13 → D8.)*
- **R4 — Topology blindness.** Accepting competitor A's version of a substation while rejecting the same session's connected line edits can break shared geometry/connectivity. Full topological validation at decision time is a project in itself. Stage-1 position: advisory warnings ("this feature's session contains N related undecided changes"), no hard constraints. Must be stated honestly as a known limitation.
- **R5 — Reviewer throughput.** With deliberate competition, most features will still have exactly one proposal. If the UI forces per-feature ceremony on those, a 500-feature session means 500 clicks and the system dies of friction. The non-competing fast path (FR-3.6) is a hard requirement, not a nicety.
- **R6 — New features can't compete by identity.** Two editors independently *creating* "the same" real-world object produce two creates with no shared feature id — competition detection by id fails. Options: spatial proximity heuristics, or (better, fits D1) an explicit **assignment/task** that both creates were made under (→ Q8, Q10). *(Decided 2026-08-13 → D16: no assignments; reviewer manual grouping + spatial-overlap suggestions; residual duplicate-create risk accepted — see doc 02 §1.)*
- **R7 — Delete vs. edit is a normal competition.** A delete proposal and an update proposal for the same feature are just two competitors; accepting delete rejects the update. No special case needed — but the compare UI must render "deletion" as a first-class side.
- **R8 — Approver value is unproven.** If the Approver never overturns reviewers, the third stage is latency, not safety. Counter-argument: a distinct commit stage gives a natural place for batch-consistent commits (R4 mitigation) and organizational sign-off. Decision needed (Q5) — see S1 for the recommendation. *(Resolved 2026-08-13 → D5: 3 stages.)*
- **R9 — Long-lived pending changes rot.** Proposals nobody reviews accumulate, drift ever further from live, and clog the queue. Need at least: an age/staleness indicator in the review queue; possibly an expiry policy (→ Q13).
- **R10 — Editor identity of truth.** Denormalized `CreatedBy/LastUpdatedBy` stamps on live rows will now reflect the *committer* pipeline, not the hand that drew the geometry, unless commit explicitly writes the original author. Small, but wrong-by-default if forgotten.

---

8. Suggestions (preliminary — proposals, not decisions)

- **S1 — Pipeline: 3 fixed stages, overlapping roles allowed.** `Draft/Edit → Review (resolve competition, technical QC) → Approve & Commit (authority sign-off, batch-consistent commit)`. Rationale: the approve stage is where R3 staleness and R4 consistency get a final gate, and where commits are batched transactionally; in small teams the same person holds both roles, so the cost is one extra click, not one extra person. If in practice approval becomes a rubber stamp, collapsing to 2 stages later is easy; adding a stage later is hard. *(Accepted 2026-08-13 → D5.)*
- **S2 — Full-state pending changes, diff-on-display.** Store the complete proposed feature (geometry + attributes + base RowVersion); compute diffs only in the UI. Simpler apply, robust against base drift, at the cost of storage — acceptable given text attributes and 2D geometries.

- **S3 — Competition as an entity, not a query.** Because competition is deliberate (D1), model it explicitly (a competition/assignment record that proposals attach to) rather than deriving it as "all pending changes with the same target id". Accidental collisions auto-create a competition on second submission. This also solves R6 for creates. *(2026-08-13, D16: the assignment half is dropped — the competition entity remains, created on id-collision or by reviewer manual grouping.)*
- **S4 — Custom history tables, not SQL Server temporal tables.** Temporal tables capture *what/when* but not *why/who-decided* (decision, reason, session, competition linkage), and they'd version live tables only — pending states never touch live tables. The decision/history model must be custom anyway; temporal tables would add a second, redundant history mechanism. Final call in Stage 3.
- **S5 — Reuse the session seam.** Client-side, a "version session" is essentially today's in-memory edit batch (`FeatureSetChangesDto`) made persistent and named. Capture at that seam; don't invent a parallel edit pipeline in the client.
- **S6 — Per-layer opt-in.** Versioning is enabled per feature table; non-versioned tables keep the current direct Sync path. Gives a pilot path (Q15) and avoids a big-bang cutover (Q17). *(Accepted 2026-08-13 → D26.)*

---

9. Open questions

Statuses: **Open** (blocks a stage), **Answered** (see §2), **Deferred**.

#	Question	Status	Notes / recommendation
Q1	Competition deliberate or accidental?	**Answered** → D1	Deliberate; accidental collisions funnel into same mechanism.
Q2	Connectivity model?	**Answered** → D2	Central DB, always online.
Q3	First target / reuse?	**Answered** → D3	Saba first, shared core.
Q4	Configurable workflow?	**Answered** → D4	Fixed. Stage count open → Q5.
Q5	How many pipeline stages, and which?	**Answered** → D5	3 stages: Edit → Review → Approve & Commit.
Q6	Session = unit of submission only; per-feature decisions?	**Answered** → D6	Yes; sessions may end partially accepted.
Q7	Can editors see each other's pending proposals?	**Answered** → D7, D18	Blind competition; own-competition status + count only.
Q8	Explicit assignment/task step creating competitions?	**Answered** → D16	**No assignment mechanics.** Collision-by-id + advisory overlap warnings. Consequences: doc 02 §1.
Q9	Delete-vs-edit competition treatment?	**Answered** → D12	Delete is an ordinary competitor.
Q10	Competing **creates** (no shared id) detection?	**Answered** → D16	No auto-competition; reviewer manual grouping, aided by spatial-overlap suggestions. Residual duplicate risk accepted (doc 02 §1).
Q11	Retention: keep history forever?	**Answered** → D13	Forever; no purge in v1.

#	Question	Status	Notes / recommendation
Q12	Undo/redo inside a draft session?	**Answered** → D14	Existing in-memory mechanism only.
Q13	Session/proposal lifetime rules?	**Answered** → D9	Flexible: multiple drafts, withdraw until first decision, no auto-expiry, age shown.
Q14	Stale proposal handling at acceptance?	**Answered** → D8	Warn + recorded override; re-checked at approval.
Q15	Pilot scope: which Saba layers/tables first?	**Answered** → D38	Transmission lines + substations.
Q16	In-app inbox (polling) acceptable for v1?	**Answered** → D15	Yes.
Q17	For versioned layers, is the current direct `Sync` write path **closed**?	**Answered** → D26	Per-layer gate; versioned layers reject direct sync.
Q18	Merge of non-overlapping competitors (geometry-only + attributes-only)?	**Deferred**	Out of stage 1 (FR-3.7); rejected editor can resubmit on top of the winner.
Q19	Multi-editor sessions?	**Answered** → D11	No — single editor per session.
Q20	Same person reviews and approves the same competition?	**Answered** → D10	Allowed; both decisions identity-stamped.
Q21	Approver-return semantics?	**Answered** → D17	Return reopens the competition; losers stay provisional until commit.
Q22	Blind-competition visibility detail?	**Answered** → D18	Status + competitor count; never rival content/authors.
Q23–Q29	Stage-2 detail questions (late arrivals during approval, no-winner closure, post-closure visibility, self-supersede, orphaned proposals, digest notifications, per-proposal withdraw)	**Answered** → D19–D25	All confirmed 2026-08-13 with the recommended defaults; see `02-workflow-and-state-machine.md` §12.
Q30–Q36	Stage-3 detail questions (draft storage, history strategy, project packaging, overlap-suggestion persistence, user references, geometry column typing, schema-signature computation)	**Answered** → D27–D33	Confirmed 2026-08-13; **Q34 modified** — display names stored at write time (D31). See `03-data-model.md` §8.
Q37–Q40	Stage-4 detail questions (post-submit client display, endpoint style, client-gateway placement, refresh model)	**Answered** → D34–D37	**Q37 and Q40 modified by Hossein** (strict query separation + on-demand pending queries with authors; no polling, manual refresh only). See `04-architecture-and-integration.md` §9.
Q41–Q44	Stage-5 detail questions (N-way attribute grid, bulk-accept confirmation, history rendering, view hosting)	**Answered** → D39–D42	All confirmed 2026-08-13 with the recommended defaults.
Q45–Q46	Stage-6 operational questions (layer-disable policy; staging environment)	**Answered** → D43–D44	Disable blocked while proposals open; current working system = staging.

10. Next steps

Planning is complete (2026-08-13): docs 01–06 baselined, D1–D44 decided, no open questions. Implementation began the same day at milestone **M1** (doc 06 §2). Any deviation discovered while building gets a new D-row here before the code merges.

STAGE 2 OF 6

Workflow & State Machine

docs/features/spatial-versioning/02-workflow-and-state-machine.md

> **Status:** BASELINED 2026-08-13 — Q23–Q29 confirmed with recommended defaults → D19–D25 (doc 01 §2)
> **Last updated:** 2026-08-13 **Prerequisite:** `01-requirements-and-boundaries.md` (decisions D1–D18; all references like D6, R1, S6 point there)

This document defines *behavior*: objects, their states, every legal transition, who may trigger it, who sees what, and which notifications fire. It does **not** define storage schema (Stage 3), API surface (Stage 4), or screen layouts (Stage 5).

Markers: **[A]** = follows directly from a D-decision. Former [P] proposals were all confirmed 2026-08-13 and now read **[A→D19]**…**[A→D25]**.

---

1. Consequences of collision-only competition (D16) — the requested discussion

Hossein chose: no assignment mechanics; editors edit whatever they have access to; the system detects competition by feature-id collision; spatial overlap produces advisory warnings. Verdict after analysis: **this is logically sound and does not break the flow**, with three consequences that must be stated honestly:

1. **"Deliberate" moves outside the system.** D1 said competition is a deliberate business process — under D16, the deliberateness is organizational (a manager verbally points two editors at the same work); the system only detects the resulting collision and cannot distinguish deliberate from accidental. That is acceptable: both were always going to funnel into the same resolution mechanism. The system's job is *detection + resolution*, not orchestration.
2. **Competing creates lose their systematic detector** (risk R6). Two editors digitizing the same new real-world object produce two creates with no shared id — no collision, no competition. Compensations, both decided in D16:
   - the reviewer can **manually group** proposals (v1 scope: create-proposals within the same layer) into one competition;
   - at submission the system runs a **spatial-overlap scan** and surfaces *suggestions* in the reviewer queue ("these 2 pending creates overlap — group them?"). **Residual risk, accepted for v1:** if the reviewer ignores the suggestion, duplicate objects can both be committed. Revisit if it happens in practice.
3. **Overlap warnings must respect blindness (D7/D18).** Two distinct channels:
   - **Editor-facing** warnings compare the edited geometry against **live** features only ("your geometry overlaps live feature #123") — live is public, no leak.
   - **Pending-vs-pending** overlaps appear **only** in the reviewer/approver queue. An editor must never learn of a rival's pending work through an overlap warning; the only competition signal an editor gets is D18's status + count on their *own* proposal.

Implementation note for Stage 3/4: the custom Maptor EF provider has no LINQ spatial predicates, so the overlap scan is raw SQL (`STIntersects` after a bounding-box pre-filter), run once per proposal at submission time — bounded cost, no interactive load.

Nothing else in the model depended on assignments. S3's "competition as an entity" survives: the entity is created lazily on second colliding submission or by manual grouping (and conceptually every submitted proposal belongs to a competition — see §2).

---

2. Object model (conceptual)

Object	One-line definition
Proposal	One proposed create/update/delete of one feature: full proposed state + base `RowVersion` + author + timestamps. (Doc 01 calls this a pending change.)
Session	Single-editor batch of proposals; unit of submission and provenance only (D6, D11).
Competition	The decision unit: the set of proposals contending for one target. **Every submitted proposal belongs to exactly one competition** — most are size 1 ("singletons"); the fast path (FR-3.6) is bulk-resolving singletons.
Decision record	Immutable record of a review/approval/return action: actor, timestamp, reason, stale-override flag. Kept forever (D13).
Notification	Inbox item (D15), aggregated per session where applicable (§8).

Two computed flags on a proposal (never states): **Stale** — base `RowVersion` ≠ current live `RowVersion`; **Orphaned** — target feature no longer exists in live.

---

3. Proposal state machine

States: `Draft`, `Submitted`, `SelectedForApproval`, `ProvisionallyRejected`, `Committed` (terminal), `Rejected` (terminal), `Withdrawn` (terminal; carries a cause: user / session-withdrawn / superseded).

#	From	To	Trigger (actor)	Guard	Side effects
P1	—	Draft	Editor adds a change to a draft session	Editor has edit access to the layer	none (private)
P2	Draft	Submitted	Session submitted (Editor)	Validation passes	Joins/creates competition (C1/C2); N1 count notices; supersedes the editor's earlier pending proposal on the same target, if any (E1) **[A→D22]**
P3	Submitted	SelectedForApproval	Reviewer selects it as winner	Its competition is Open; if Stale → recorded override (D8)	Competition → Resolved; all siblings → ProvisionallyRejected (P4)
P4	Submitted	ProvisionallyRejected	Sibling selected as winner	—	Rejection reason recorded now; notification deferred to commit (D17)
P5	Submitted	Rejected	Reviewer closes competition with **no winner** **[A→D20]**	Competition Open	Competition → ClosedNoWinner; N4 fires immediately (final)

#	From	To	Trigger (actor)	Guard	Side effects
P6	SelectedForApproval	Committed	Approver approves (commit transaction)	Re-validation: schema still valid (R1 gate), staleness re-checked — if newly stale → approver override (D8)	Live write (original author stamped, R10); history + decision records; siblings → Rejected (P9); N2/N3 digests
P7	SelectedForApproval	Submitted	Approver returns with reason (D17)	—	Competition → Open; siblings → Submitted (P10); N5 to reviewer
P8	Submitted	Withdrawn	Editor withdraws proposal **[A→D25]** / withdraws session / is superseded (E1)	Its competition is still Open	Leaves competition; competition dissolves if now empty (C5)
P9	ProvisionallyRejected	Rejected	Competition committed	—	N3 (reason recorded at P4)
P10	ProvisionallyRejected	Submitted	Approver returns (D17)	—	back in play

A Draft proposal that is simply deleted from its draft session ceases to exist — no state, no record (D14: drafts are the editor's private workspace).

---

4. Competition state machine

States: `Open`, `Resolved`, `Committed` (terminal), `ClosedNoWinner` (terminal), `Dissolved` (terminal — emptied by withdrawals, no decisions were made).

#	From	To	Trigger	Guard	Notes
C1	—	Open	First proposal submitted for a target with no open competition	—	Size 1 (singleton)
C2	Open	Open	Colliding submission joins; or reviewer manually groups proposals/competitions (D16)	Still Open; manual grouping v1 = creates within same layer	N1 count notices to all owners
C3	Open	Resolved	Reviewer selects winner	≥1 proposal; stale override if needed (D8)	Drives P3/P4
C4	Open	ClosedNoWinner	Reviewer rejects **all** proposals	—	No live change ⇒ no approver gate **[A→D20]**; drives P5
C5	Open	Dissolved	Last proposal withdrawn	Size 0	No decision records
C6	Resolved	Committed	Approver approves	Commit transaction succeeds (E9)	Drives P6/P9
C7	Resolved	Open	Approver returns with reason	—	Drives P7/P10; return reason visible to reviewer

Late arrivals [A→D19]: while a competition on target F is `Resolved` (sitting in the approval queue), a new submission targeting F does **not** join or reopen it — it opens a new, *queued* competition on F. Guard: a queued competition cannot be Resolved until its predecessor reaches a terminal state. At most one Resolved + one Open competition per target can exist. (Alternative — late arrival reopens the Resolved competition — was rejected: it would make the approval queue unstable under editor activity.)

5. Session state machine

Stored states: `Draft` → `Submitted`; `Withdrawn` (terminal). Everything else is **derived, read-only** (D6): *InReview* (any proposal undecided), *PartiallyResolved* (mixed outcomes so far), *Resolved* (all proposals terminal).

Rules: editable only in Draft **[A]**; submit is one-way **[A]**; withdraw allowed while no contained proposal has left `Submitted` (D9) and withdraws all its proposals (P8); multiple concurrent drafts per editor (D9); exactly one editor per session (D11).

6. Roles and permitted actions

New permission-enum group (names indicative, final in Stage 3/4): `VersionEdit`, `VersionReview`, `VersionApprove`, `VersionHistoryRead` — composed into Saba roles via the existing `Role` / `RolePermission` model (FR-4.3). One person may hold several (D10).

Action	Editor	Reviewer	Approver	Viewer
Create/edit/discard draft sessions (own; within layer access)	✓	—	—	—
Submit session / withdraw session (guards §5)	✓	—	—	—
Withdraw own pending proposal **[A→D25]**	✓	—	—	—
See own proposals' status + competitor count (D18)	✓	—	—	—
See review queue, all pending content + authors	—	✓	✓	—
Compare competitors side-by-side (geometry + attribute diff)	—	✓	✓	—
Select winner / reject-all (with stale override, D8)	—	✓	—	—
Manually group proposals into a competition (D16)	—	✓	—	—
Approve & commit (single or batch), with re-validation override	—	—	✓	—
Return a Resolved competition with reason (D17)	—	—	✓	—
Read live state + committed history	✓	✓	✓	✓
Receive inbox notifications	✓	✓	✓	—

7. Visibility matrix (blind rules)

Artifact	Owner-editor	Other editors	Reviewer	Approver	Viewer
Draft session content	✓	—	—	—	—
Own submitted proposal (content, status, competitor **count**)	✓	—	✓	✓	—
Rival pending proposals (content, authors)	—	—	✓	✓	—
Spatial-overlap warnings vs **live** features	✓ (own edits)	—	✓	✓	—

Artifact	Owner-editor	Other editors	Reviewer	Approver	Viewer
Pending-vs-pending overlap suggestions	—	—	✓	✓	—
Live features	✓	✓	✓	✓	✓
Committed history (winning versions + commit decisions)	✓	✓	✓	✓	✓
Full record of a **closed** competition (incl. losing proposals, authors, reasons)	**[A→D21]** own outcome always; full record proposed for participants + reviewer/approver only	— **[A→D21]**	✓	✓	— **[A→D21]**

Q25 default (confirmed → D21): blindness protects the *pending* phase; after closure, each participant sees the full record of the competitions they took part in (including the winner's content — that is now committed history anyway); non-participants see only committed history. Widening to full transparency is a policy switch later, not a redesign.

Amendment 2026-08-13 (D34): rival *authors* are discoverable on demand via the per-feature pending-status check (count + authors), even before the editor submits their own proposal — the author cells marked "—" for editors above are superseded accordingly. Rival *content* cells stand: content stays hidden until closure. Additionally, editor-facing status labels must collapse the provisional states (`SelectedForApproval` / `ProvisionallyRejected` both render as "under review"), so D17's deferred rejection notification is not defeated by the My-Pending panel.

---

8. Notifications (in-app inbox, polling — D15)

Digest rule [A→D24]: notifications aggregate per (session × event type × batch) — a 500-feature session produces "480 committed, 15 rejected, 5 still pending" with expandable detail, not 500 inbox rows. Required by reviewer/editor throughput (R5).

#	Event	Recipients	Content
N1	Proposal enters / gains competitors	Owners of all proposals in that competition	"Your proposal for (feature) now competes with N−1 others" — count only (D18)
N2	Winner committed	Winning editor	Committed confirmation (digest)
N3	Final rejection (at commit — D17)	Each losing editor	Rejection reason recorded at review time (digest)
N4	Competition closed with no winner	All owners	Reasons (immediate — nothing provisional remains)
N5	Approver return	The reviewer who resolved	Return reason
N6	Proposal became Orphaned (target deleted in live)	Owner + reviewer queue badge	Advisory; see E2

Withdrawals notify no one (the actor already knows; queues update silently). Staleness is a queue badge, not an inbox event — it can flip repeatedly and would spam.

9. End-to-end walkthroughs

A — Deliberate competition, happy path. Editors E1, E2 are (verbally) pointed at feature F. Each drafts a session, edits F, submits. E1's submission creates a singleton competition (C1); E2's joins it (C2) — both get N1 ("competing with 1 other"), neither sees the rival's work. Reviewer opens the compare view (geometry + attribute diff, authors visible), selects E2 (P3); E1 → ProvisionallyRejected with reason (P4). Approver sees the Resolved competition, re-validation passes, approves (C6): commit writes E2's state to live stamped with E2 as author, E1 → Rejected, N2 to E2, N3 to E1.

B — Bulk session, fast path. Editor submits 500 proposals; 498 become singletons, 2 join existing competitions. Reviewer bulk-selects the 498 singletons → accept-all (498 × P3 in one action); the 2 contested ones go through compare. Approver batch-approves → one transaction (E9), one digest each way.

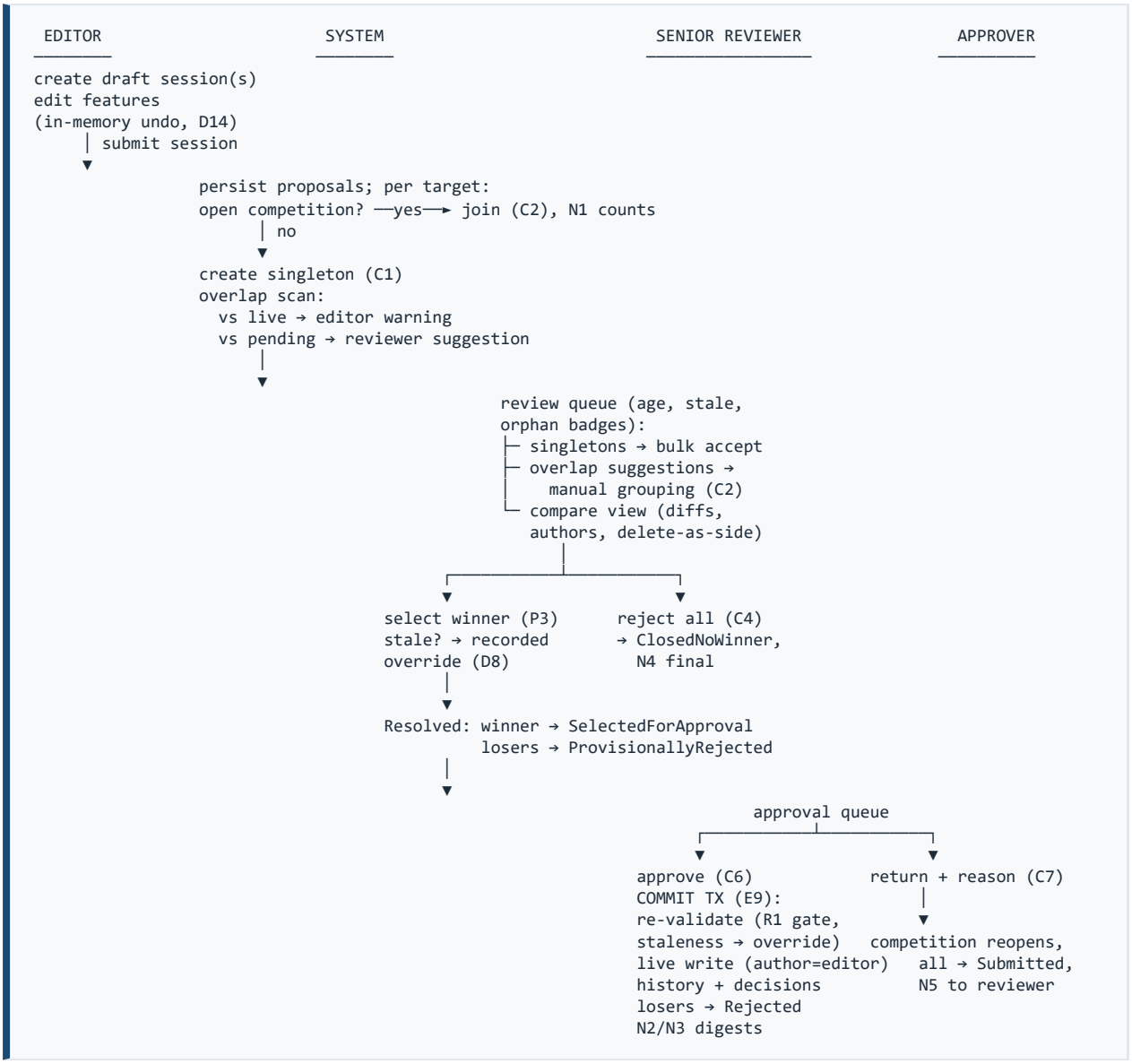
C — Stale winner. Competition on F resolved for E1; before approval, a queued competition's winner on neighboring feature G commits and — separately — F itself was committed last week by another competition, so E1's base is old. Reviewer already recorded a stale override at selection (D8). At approval, staleness is re-checked: live moved *again* since review → approver sees a fresh warning and must override again (or return). Both overrides are in the decision records.

D — Approver return. Approver disagrees with the winner, returns with reason (C7). All proposals revert to Submitted, competition is Open again, reviewer gets N5, losers never received a rejection notice (D17) — nothing was wasted. Reviewer re-resolves (possibly picking a previous loser).

E — Delete vs edit (D12). E1 proposes deleting F; E2 proposes an update. Same competition; compare view renders "deletion" as one side. Accepting the delete commits a live delete; E2's update → Rejected. All other pending proposals on F become Orphaned (E2 flag).

F — Competing creates (D16). E1 and E2 both digitize the same new substation. Two singleton competitions — no id collision. The submission-time overlap scan flags the pair in the reviewer queue; reviewer manually groups them (C2) into one competition and resolves normally. If the reviewer skips grouping, both commit — accepted v1 risk (§1).

10. Overall flow (text flowchart)



11. Edge cases & rules

- **E1 — Self-supersede [A→D22].** One active proposal per (editor, target). Submitting a new proposal for a target where the editor already has a pending one auto-withdraws the old (`Withdrawn`, cause=superseded). No notification. Rationale: an editor competing with themself is noise; "my latest word on this feature" is the intuitive contract.
- **E2 — Orphaned proposals [A→D23].** If the target is deleted in live (a delete committed), remaining pending update-proposals on it are flagged Orphaned (N6). In v1 they can only be rejected (with an automatic system note); "accept as re-create" is out of scope — the editor resubmits as a create if still relevant.
- **E3 — Schema migration with pending proposals (R1).** Commit (P6) re-validates the serialized attributes against the current schema; mismatch blocks the commit with a clear error → approver returns, reviewer/editor deal with it. An EF-migration checklist item ("check pending-proposal compatibility") is part of the ops docs. Details Stage 3.

- **E4 — Concurrent reviewers.** Two reviewers resolving the same competition: optimistic concurrency on the competition row; second save fails with a "already resolved" error. Same for two approvers on one competition.
- **E5 — Withdraw guards.** Session withdraw: only while all its proposals are still `Submitted` / `Draft` (D9). Proposal withdraw: only while its competition is Open — a winner cannot withdraw during approval (await commit or return).
- **E6 — Deactivated editor.** Pending proposals remain and are decided normally; notifications accumulate in the inactive inbox. No special handling in v1.
- **E7 — Queued competitions [A→D19].** See §4 late-arrival rule. When the predecessor commits, the queued competition's proposals are typically Stale by construction — handled by the normal D8 override flow, not a special case.
- **E8 — Blind overlap channels.** Restated from §1: editor warnings reference live features only; pending-vs-pending only ever surfaces to reviewer/approver.
- **E9 — Batch commit failure.** A batch approval is one transaction: if any competition in it fails re-validation, nothing commits; failures are listed; the approver deselects the failing ones and retries. Guarantees NFR-3 (all-or-nothing per approved batch).

---

12. Stage-2 detail questions (Q23–Q29) — all ANSWERED 2026-08-13

All confirmed with the recommended defaults → decisions **D19–D25** in doc 01 §2. Table kept for the rationale/impact notes.

#	Question	Recommendation	Impact if changed
Q23	Late submissions while a competition is in approval → new **queued** competition (max one Resolved + one Open per target)?	Yes (§4)	Affects Stage-3 uniqueness constraints ("one open competition per target").
Q24	Reviewer may close a competition **with no winner** (all finally rejected) without approver involvement, since live is untouched?	Yes (C4/P5)	If approver oversight is wanted even for no-winner, add a Resolved-NoWinner state + gate.
Q25	Post-closure visibility: participants see the full record of their own competitions; non-participants see committed history only?	Yes (§7)	Pure policy; widening later is trivial.
Q26	Self-supersede: one active proposal per (editor, target); newer submission auto-withdraws the older?	Yes (E1)	Affects a Stage-3 unique index.
Q27	Orphaned proposals (target deleted in live) can only be rejected in v1 (no accept-as-recreate)?	Yes (E2)	Accept-as-recreate needs id-reuse semantics — real design work.
Q28	Notifications aggregate as per-session digests?	Yes (§8)	Without it, bulk sessions spam inboxes (R5).
Q29	Editors may withdraw an individual pending proposal (not just the whole session), while its competition is Open?	Yes (P8/E5)	Removing it simplifies guards slightly, at UX cost.

13. Handoff to Stage 3 (data model)

Entities implied by this document: `Proposal` (polymorphic target: layer/table id + feature id + serialized state + base RowVersion), `Session`, `Competition` (+ membership), `DecisionRecord`, `Notification`, per-layer versioning flag (S6/Q17), committed-history store. Constraints to carry: one open competition per target (Q23), one active proposal per editor+target (Q26), optimistic concurrency on competition (E4). Unresolved before Stage 4/6: Q15 (pilot layers), Q17 (closing direct Sync). Biggest open design problem for Stage 3: the R1 schema-drift gate and the shape of the committed-history tables (S4: custom, not temporal).

STAGE 3 OF 6

Data Model

`docs/features/spatial-versioning/03-data-model.md`

> **Status:** BASELINED 2026-08-13 — Q30–Q36 confirmed → D27–D33 (Q34 in modified form: display names stored, see D31); Q17 → D26 **Last updated:** 2026-08-13 **Prerequisites:** `01-requirements-and-boundaries.md` (D1–D25), `02-workflow-and-state-machine.md` (states, transitions, constraints in its §13)

This document defines *storage*: entities, columns, constraints, indexes, the history strategy, the commit algorithm, and the schema-drift gate. It does not define API endpoints (Stage 4) or UI (Stage 5). Field names/types are the proposal to build EF Core configurations from — refinement during implementation is expected; *structural* changes go back through the decision log.

Markers: **[A]** = follows from a D-decision. Former [P] proposals were confirmed 2026-08-13 and now read **[A→D27]**…**[A→D33]**; Q34 was accepted in **modified** form (D31: display names stored at write time, not resolved at query time).

---

1. Design principles

1. **Polymorphic, serialized proposals [A: R1/D16].** One generic proposal store serves all ~100 feature tables: target = (versioned-layer id, feature id), proposed state = geometry column + attributes JSON. No per-table pending tables.
2. **Copy-on-write history [A→D28].** No baseline snapshot when a layer opts in. At every commit, the live row being overwritten/deleted is first copied into `FeatureHistory`. History size is proportional to change volume, not data volume; time-travel = walk the chain backward from live. Pre-versioning history simply doesn't exist (acceptable: the audit obligation starts when versioning starts).
3. **Append-only decisions [A: D13].** `DecisionRecord` rows are never updated or deleted.
4. **Shared-library clean [A: D3].** Core entities live in Maptor shared projects and reference Saba users as scalar ids **plus a display name stamped at write time** [A→D31]; all versioning tables sit in their own SQL schema `versioning`, away from the feature tables.
5. **Provider reality [A: §6 of doc 01].** Geometry uses the custom Maptor provider (no NTS). Proposal/history geometry columns are typed as SQL Server `geometry` (pass-through mapping) rather than `varbinary` [A→D32] — this enables spatial indexes and raw-SQL `STIntersects` for the overlap scan. Spatial indexes need a BOUNDING_BOX: per-layer extent from `VersionedLayer`, defaulting to the Iran country extent.
6. **Flags are computed, not stored.** Stale (base RowVersion ≠ live) and Orphaned (live row gone) are computed in queue queries by joining live tables — they can never go out of date.

---

2. Entity catalog

All tables in schema `versioning`. `RowVersion` columns are SQL `rowversion` concurrency tokens. All timestamps UTC.

2.1 VersionedLayer — per-layer opt-in registry [A: S6]

Column	Type	Notes
Id	int PK identity	
LayerKey	uniqueidentifier, unique	Matches client `FeatureSet.LayerId`
EntityName	nvarchar(200), unique	Server entity/table identity (e.g. `TransmissionLine`)
DisplayName	nvarchar(200)	
IsVersioningEnabled	bit	Gate for the Sync path (Q17, Stage 4)
SchemaSignature	nvarchar(100)	Current schema hash, §5; recomputed at API startup [A→D33]
SchemaSignatureUpdatedAt	datetime2	
ExtentMinX/MinY/MaxX/MaxY	float, nullable	Spatial-index bounding box; null → country default
EnabledAt	datetime2	History starts here (principle 2)

2.2 VersionSession — submission batch [A: D6, D9, D11]

Column	Type	Notes
Id	bigint PK identity	
EditorUserId	bigint	Scalar, no FK [A→D31]
EditorDisplayName	nvarchar(200)	Stamped at submission [A→D31]
Title	nvarchar(200), null	
Comment	nvarchar(1000), null	
State	tinyint	Submitted=1, Withdrawn=2 — **no Draft row [A→D27]**: drafts live client-side; the session row is created at submission
SubmittedAt	datetime2	
WithdrawnAt	datetime2, null	
RowVersion	rowversion	

Index: (EditorUserId, State). Derived statuses (InReview/PartiallyResolved/Resolved) are queries over proposals, never columns [A: D6].

2.3 Proposal — one proposed create/update/delete [A: FR-1]

Column	Type	Notes
Id	bigint PK identity	
SessionId	bigint FK → VersionSession	

Column	Type	Notes
EditorUserId	bigint	Denormalized from session — needed for the D22 unique index and visibility queries
EditorDisplayName	nvarchar(200)	Stamped at submission [A→D31]
VersionedLayerId	int FK → VersionedLayer	
TargetFeatureId	bigint, null	Null for creates
ClientKey	uniqueidentifier	From `Feature<T>.Key`; the create's pre-commit identity (candidate mapping to live `gis_id` — note in §5.2)
ChangeType	tinyint	Create=1, Update=2, Delete=3
ProposedGeometry	geometry, null	Null for Delete [A→D32]
ProposedAttributesJson	nvarchar(max), null	Null for Delete; contract in §5.1
BaseRowVersion	binary(8), null	Null for Create [A: FR-1.4]
SchemaSignatureAtSubmit	nvarchar(100)	R1 gate input
CompetitionId	bigint FK → Competition	Every submitted proposal belongs to exactly one (doc 02 §2)
State	tinyint	Submitted=0, SelectedForApproval=1, ProvisionallyRejected=2, Committed=3, Rejected=4, Withdrawn=5 — active states deliberately ≤2 for filtered indexes
WithdrawCause	tinyint, null	User=1, SessionWithdrawn=2, Superseded=3 [A: D22/D25]
SubmittedAt / DecidedAt / FinalizedAt	datetime2 (last two null)	
RowVersion	rowversion	

Constraints & indexes:

- **D22 (self-supersede backstop):** unique filtered index on (VersionedLayerId, TargetFeatureId, EditorUserId) `WHERE State <= 2 AND TargetFeatureId IS NOT NULL`. The service layer performs the supersede; the index makes races lose loudly.
- Collision lookup: index (VersionedLayerId, TargetFeatureId) `WHERE State <= 2`.
- Index CompetitionId; index SessionId.
- Spatial index on ProposedGeometry (overlap scan) [A→D32].

2.4 Competition — the decision unit [A: S3, doc 02 §4]

Column	Type	Notes
Id	bigint PK identity	
VersionedLayerId	int FK	
TargetFeatureId	bigint, null	Null for manually-grouped create competitions [A: D16]
Kind	tinyint	IdCollision=1, ManualGroup=2

Column	Type	Notes
State	tinyint	Open=0, Resolved=1, Committed=2, ClosedNoWinner=3, Dissolved=4
WinnerProposalId	bigint, null, FK → Proposal	Set at Resolved. Circular FK with Proposal.CompetitionId — configure both without cascade delete
PredecessorCompetitionId	bigint, null, self-FK	Queued chain [A: D19]
CreatedAt / ResolvedAt / FinalizedAt	datetime2 (last two null)	
RowVersion	rowversion	Guards E4 (concurrent reviewers/approvers)

Constraints (**D19**): two unique filtered indexes on (VersionedLayerId, TargetFeatureId): one `WHERE State = 0 AND TargetFeatureId IS NOT NULL`, one `WHERE State = 1 AND TargetFeatureId IS NOT NULL` — at most one Open and one Resolved competition per feature, enforced by the database itself.

2.5 DecisionRecord — append-only audit of every action [A: FR-5.1, D13]

Column	Type	Notes
Id	bigint PK identity	
CompetitionId	bigint FK	
ProposalId	bigint, null, FK	Null for competition-level actions (Return)
ActorUserId	bigint	
ActorDisplayName	nvarchar(200)	Stamped at decision time [A→D31]
Action	tinyint	SelectWinner=1, RejectProposal=2 (one row per loser, with reason), CloseNoWinner=3, Approve=4, Return=5
Reason	nvarchar(1000), null	Required by service for Reject/CloseNoWinner/Return [A: FR-3.5, D17]
IsStaleOverride	bit	Set on SelectWinner/Approve when D8 override was exercised
CommitBatchId	bigint, null, FK	Set on Approve
CreatedAt	datetime2	

No UPDATE/DELETE ever (enforce by convention + optionally a DENY or trigger later). Indexes: CompetitionId; ActorUserId.

2.6 CommitBatch — one approval transaction [A: NFR-3, E9]

Column	Type	Notes
Id	bigint PK identity	
ApproverUserId	bigint	
ApproverDisplayName	nvarchar(200)	Stamped at commit [A→D31]
CommittedAt	datetime2	

Column	Type	Notes
CompetitionCount	int	Convenience

2.7 FeatureHistory — copy-on-write past states [A→D28]

Written inside the commit transaction, *before* live is overwritten or deleted. Creates write no history row (nothing was replaced).

Column	Type	Notes
Id	bigint PK identity	
VersionedLayerId	int FK	
FeatureId	bigint	
Geometry	geometry	The state being replaced [A→D32]
AttributesJson	nvarchar(max)	Serialized with the same contract (§5.1)
ReplacedRowVersion	binary(8)	The RowVersion this state had
CommitBatchId	bigint FK	Which commit replaced it
WinningProposalId	bigint FK → Proposal	What replaced it (author, session, competition all reachable from here)
SupersededAt	datetime2	= batch CommittedAt

Index: (VersionedLayerId, FeatureId, SupersededAt DESC). Answering FR-5.3 ("what did F look like before commit X"): the row with CommitBatchId = X; full timeline: live row + chain of history rows descending. A committed *delete* leaves the last state here with the delete-proposal as its replacer — nothing is ever lost.

2.8 VersionNotification — in-app inbox [A: D15, D24]

Column	Type	Notes
Id	bigint PK identity	
RecipientUserId	bigint	
Type	tinyint	N1..N6 from doc 02 §8
SessionId / CompetitionId	bigint, null	Digest anchors
PayloadJson	nvarchar(max)	Counts, reasons, feature refs — expandable detail
CreatedAt / ReadAt	datetime2 (ReadAt null)	

Digests [A: D24] are formed at write time: one row per (recipient × session × event batch). Index: (RecipientUserId, ReadAt).

2.9 OverlapSuggestion — reviewer grouping aid [A→D30]

Computed once at submission (raw-SQL bbox + `STIntersects`), persisted so the reviewer queue is cheap to render and suggestions are dismissable.

Column	Type	Notes
Id	bigint PK identity	
ProposalId	bigint FK	The newly submitted proposal
OverlapKind	tinyint	PendingVsPending=1 (reviewer-only, blind rule E8), PendingVsLive=2
OtherProposalId	bigint, null, FK	For kind 1
LiveFeatureId	bigint, null	For kind 2
ComputedAt	datetime2	
DismissedByUserId / DismissedByDisplayName / DismissedAt	bigint / nvarchar(200) / datetime2, null	Reviewer "not the same object" [A→D31]

3. Relationships (summary)

```
VersionedLayer 1──* Proposal            VersionSession 1──* Proposal
VersionedLayer 1──* Competition         Competition    1──* Proposal (CompetitionId)
Competition    0..1──1 Proposal (WinnerProposalId)     Competition 0..1──0..1 Competition (Predecessor)
Competition    1──* DecisionRecord      CommitBatch    1──* DecisionRecord (Approve rows)
CommitBatch    1──* FeatureHistory      Proposal       1──* FeatureHistory (WinningProposalId)
Proposal       1──* OverlapSuggestion   (users: scalar ids + stored display names, no FKs [A→D31])
```

4. Enums (shared core)

`ProposalChangeType` {Create, Update, Delete} · `ProposalState` {Submitted, SelectedForApproval, ProvisionallyRejected, Committed, Rejected, Withdrawn} · `WithdrawCause` {User, SessionWithdrawn, Superseded} · `SessionState` {Submitted, Withdrawn} · `CompetitionState` {Open, Resolved, Committed, ClosedNoWinner, Dissolved} · `CompetitionKind` {IdCollision, ManualGroup} · `DecisionAction` {SelectWinner, RejectProposal, CloseNoWinner, Approve, Return} · `NotificationType` {N1..N6}.

Client-side `FeatureStatus` (existing, in `Sta.Common`) stays untouched — it describes the in-memory draft; `ProposalState` starts where the draft ends [A: Q30 model].

5. Key mechanics

5.1 Serialization contract

- **Attributes JSON:** flat object, keys = field names, values as JSON natives; dates ISO 8601 UTC; numbers invariant-culture; keys sorted ordinally (canonical form → diffable, hashable). Excluded: `RowVersion` (own column), identity/computed columns, the audit stamps `CreatedBy*/LastUpdated*` (assigned at commit, R10).
- **Geometry:** the provider's native SQL Server binary, stored in a `geometry` column [A→D32]; SRID preserved; same bytes the map stack already reads/writes.

- **SchemaSignature:** SHA-256 (hex, truncated 32 chars) over the ordered list of `FieldName:StoreType:IsNullable` for the entity's non-excluded columns. Computed from EF model metadata at API startup and stamped into `VersionedLayer` [A→D33] — no manual migration step to forget; a changed signature after deploy is *detected*, not declared.

5.2 Commit transaction (approver, batch — implements P6/C6/E9)

One DB transaction per `CommitBatch`:

1. Load the batch's competitions with `RowVersion` check — all must still be `Resolved`.
2. Per winner: load the live row. **Stale gate [A: D8]:** live RowVersion ≠ proposal's BaseRowVersion and no covering override recorded at approve time → abort batch, report. **Schema gate [A: R1/E3]:** `SchemaSignatureAtSubmit` ≠ layer's current → attempt tolerant mapping (§5.3); unresolvable → abort batch, report.
3. Copy-on-write: Update/Delete → insert `FeatureHistory` from the live row.
4. Apply: Create → insert live row, `CreatedBy` = **editor id + display name** [A: R10, D31] — matching the existing `FeatureBaseEntity.CreatedByFullName` convention (note: candidate — stamp `gis_id` from ClientKey, resolve in implementation); Update → overwrite live, `LastUpdatedBy*` = editor id + name; Delete → delete live row.
5. Write `CommitBatch`, Approve `DecisionRecord`s, state flips (winner → Committed, provisionally rejected → Rejected [A: D17]), competition → Committed.
6. Write digest notifications (N2/N3).
7. `SaveChanges` — any failure rolls back everything [A: E9].

Live-table writes go through the same `BargContext` (same transaction) using the entity types — no raw SQL needed for the apply step; the JSON→entity mapping reuses the sync pipeline's attribute-dictionary mapping.

5.3 Schema-drift tolerant mapping [A: E3]

At review-display and at commit, when signatures differ, diff the field lists:

- **Added nullable column** since submit → apply with NULL — warning, allowed.
- **Added non-nullable (no default)** → block.
- **Removed column** → drop the value — warning, allowed.
- **Type change / rename** (appears as remove+add) → block; reviewer/approver sees which fields; resolution = return/reject, editor resubmits under the new schema. Blocks are per-proposal but abort the whole batch (E9); the approver retries without the blocked competitions.

5.4 Time-travel queries

`GetFeatureAsOf(layer, featureId, dateTime)`: latest `FeatureHistory` row with `SupersededAt > dateTime` gives the state *at* that time (or the live row if none). `GetFeatureTimeline(layer, featureId)`: history rows ascending + live; each hop links its `WinningProposalId` → author, session, competition, decisions. Serving FR-5.3 for audit/inspection UI — mass time-slice *map rendering* is explicitly not a v1 target.

5.5 Overlap scan (submission-time, raw SQL)

For each submitted proposal with geometry: one query against pending proposals of the same layer (`State <= 2`, other editors) and one against the live table, both `bbox-filter AND`

`geometry.STIntersects(@g) = 1`. Results → `OverlapSuggestion` rows [A→D30]. Kind 1 feeds the reviewer queue (E8 blind rule); kind 2 returns in the submission response as the editor-facing advisory (§1 of doc 02).

---

6. EF Core & project integration

- **New shared projects [A→D29]:**
  - `IRI.Maptor.Sta.Versioning` (netstandard2.1): entities, enums, state-machine guard logic (pure functions: may-transition checks), serialization contract, signature computation — no EF dependency.
  - `IRI.Maptor.Ket.VersioningPersistence`: `IEntityTypeConfiguration`s, the `modelBuilder.AddMaptorVersioning(schema: "versioning")` extension, repositories/ services (submission, collision/competition service, review service, commit service, overlap scan).
- **Saba integration:** `BargContext` calls `AddMaptorVersioning()`; one EF migration adds the `versioning` schema. Controllers/handlers (Stage 4) sit in the existing Presentation/Application layers. WPF UI (Stage 5) goes into Jab per doc 01 FR/Stage 5.
- **User references:** scalar `…UserId` columns plus explicit `…DisplayName` columns stamped at write time; no FKs to Saba's Users table [A→D31]. Names are historical facts, not lookups — decision history outlives any user-account lifecycle and renders without joins.
- **Migrations note (R1):** the versioning tables themselves are ordinary EF migrations; the *feature-table* migration checklist gains one automatic behavior — signatures are recomputed at startup [A→D33] and pending proposals against changed layers get flagged in the review queue immediately.

---

7. Sizing & performance notes

- Proposal row ≈ 1–10 KB (JSON) + geometry. Even pessimistic pilot use (10 editors × 500 features/day) is ~50 MB/month — negligible for SQL Server.
- History grows only with commits (copy-on-write) — proportional to real change, forever retention (D13) is viable; revisit only if a bulk-recapture project lands.
- Queue queries join live tables per layer for stale/orphan flags — indexed PK joins, fine at pilot scale; if the pending set ever reaches 10^5+, add a cached flag refreshed on commit (not now).
- Spatial index on `Proposal.ProposedGeometry` keeps the overlap scan sub-second; it runs once per submission, never interactively.

---

8. Stage-3 detail questions (Q30–Q36) — all ANSWERED 2026-08-13

Confirmed with the recommended defaults → **D27–D33** (doc 01 §2), except **Q34 accepted in modified form**: display names are stored explicitly at write time alongside the ids (D31), not resolved at query time. Table kept for rationale/impact notes.

#	Question	Recommendation	Impact if changed
Q30	Draft sessions are **client-side only** — the server session row is created at submission (no server-side draft autosave)?	Yes — matches D14 and today's edit flow; a crash loses only the local draft, as it does today	Server-side drafts = an autosave/sync subsystem: chattier API, draft privacy handling, real scope growth
Q31	History = **copy-on-write at commit**, no baseline snapshot at layer enablement (history starts empty at EnabledAt)?	Yes — storage ∝ change volume; baseline of ~100 tables would duplicate the whole DB	Baseline gives pre-versioning time-travel at heavy storage + capture-job cost
Q32	New shared projects `IRI.Maptor.Sta.Versioning` (model, no EF) + `IRI.Maptor.Ket.VersioningPersistence` (EF + services), tables in SQL schema `versioning`?	Yes — mirrors the existing Sta/Ket split (D3)	Building inside Saba first = faster start, extraction debt later
Q33	Overlap suggestions **persisted** (dismissable rows, computed once at submission) rather than computed on the fly in the queue?	Yes — stable, cheap queue, dismissals remembered	On-the-fly = no table, but repeated spatial queries per queue load and no dismiss memory
Q34	User references as **scalar ids only** (no FK to Saba Users), names resolved at query time?	Yes — shared-lib clean, history survives account lifecycle	FKs give referential comfort but couple core tables to Saba and complicate user archival
Q35	Proposal/history geometry columns typed **SQL `geometry`** (spatial index + STIntersects) instead of varbinary?	Yes — the pass-through mapping already exists; overlap scan needs it	varbinary = no spatial queries on pending data; overlap scan would need client-side filtering
Q36	SchemaSignature **computed automatically from the EF model at API startup** and stamped into VersionedLayer?	Yes — drift is detected, never declared; no manual step to forget	Manual/migration-time stamping risks the exact silent failure R1 warns about

9. Handoff to Stage 4 (architecture & integration)

Q17 is answered → **D26** (per-layer gate; the `IsVersioningEnabled` flag in §2.1 anchors it) and Q30–Q36 → D27–D33. Stage 4 then covers: API surface (submission, queues, compare payloads, decisions, commit, inbox), where competition/diff logic runs (server vs client), Jab.Wpf hosting of the versioning views vs a new UI library, and the authorization mapping of the four `Version*` permissions into Saba's Permission enum. Q15 (pilot layers) stays open until Stage 6.

STAGE 4 OF 6

Architecture & Integration

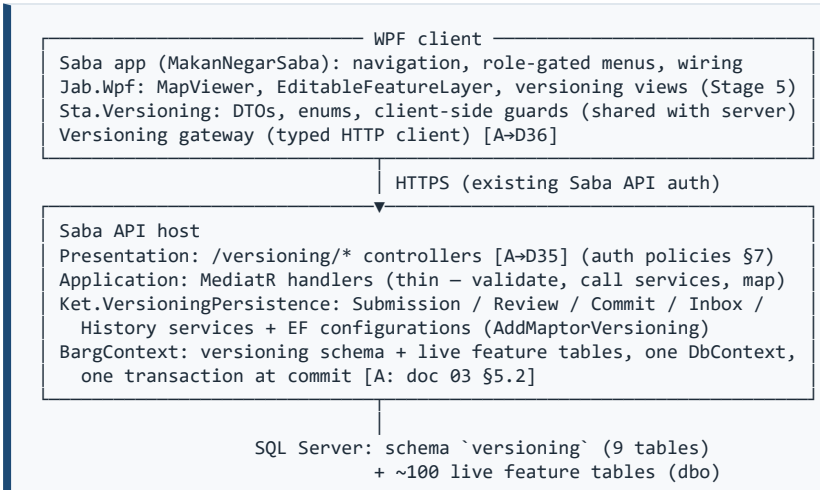
docs/features/spatial-versioning/04-architecture-and-integration.md

> **Status:** BASELINED 2026-08-13 — Q37–Q40 answered (Q37 and Q40 in modified form) → D34–D37; pilot layers → D38 **Last updated:** 2026-08-13 **Prerequisites:** docs 01–03 (decisions D1–D33)

This document defines *where things live and how they talk*: project layout, the Saba API surface, the Sync-path gate, client integration seams, authorization mapping, and cross-cutting concerns. Screen design is Stage 5; milestones are Stage 6.

Markers: **[A]** = follows from a D-decision. Former [P] proposals were answered 2026-08-13 → **[A→D34]**…**[A→D37]**; Q37 and Q40 were accepted in **modified** form (D34: strict query separation + on-demand pending queries revealing count **and authors**; D37: no polling — manual refresh only).

1. Component overview



The versioning core never references Saba types [A: D3]; Saba references the core.

2. Project layout and boundaries [A: D29]

Project	Contains	Must NOT contain
`IRI.Maptor.Sta.Versioning` (new, netstandard2.1)	Entities, enums, state-machine guard functions (pure: `CanWithdraw(proposal)`, `CanResolve(competition)` …), serialization contract + schema-signature computation, **DTOs** (submission, queue rows, compare payload, decision commands, inbox items)	EF, HTTP, UI, Saba types

Project	Contains	Must NOT contain
`IRI.Maptor.Ket.VersioningPersistence` (new)	`IEntityTypeConfiguration`s, `modelBuilder.AddMaptorVersioning(schema)`, services: SubmissionService (validation, collision → competition, supersede D22, overlap scan D30/D32), ReviewService (queues, select/reject/close, manual grouping), CommitService (doc 03 §5.2), InboxService (digests D24), HistoryService (timeline/as-of)	Controllers, Saba types, UI
Versioning gateway [A→D36]	Typed HTTP client wrapping the /versioning API; `VersioningWebDataSource : IEditableVectorDataSource` for the save-seam (§4)	UI
`IRI.Maptor.Jab.Wpf`	Versioning view models + views (Stage 5), reusing `FeatureChangesViewModel` diff mechanics	Direct DB access
Saba API (Presentation/Application)	Thin controllers + MediatR handlers; user-context injection (id + display name, D31); layer-registry admin	Business rules (those live in services/guards)
Saba WPF app	Navigation, role-gated menus, gateway registration	Versioning logic

Guard functions live in `Sta.Versioning` so client UI and server services enforce the *same* transition rules — the server remains the authority; the client uses them only to enable/disable actions.

---

3. API surface (v1 sketch)

Dedicated `/versioning/*` controllers [A→D35], MediatR behind each. Exact DTO shapes are implementation detail; semantic content is fixed by docs 02–03.

Endpoint	Auth (§7)	Purpose
`GET /versioning/layers`	any versioning role	Registry: which layers versioned (client routes saves, §4)
`POST /versioning/sessions`	VersionEdit	Submit a session (proposals batch). Returns per-proposal: state, competition status + count (D18), editor-facing live-overlap advisories (kind 2, D30), supersede notices (D22)
`DELETE /versioning/sessions/{id}`	VersionEdit (owner)	Withdraw session (guard D9)
`DELETE /versioning/proposals/{id}`	VersionEdit (owner)	Withdraw one proposal (guard D25)
`GET /versioning/my/sessions`, `GET /versioning/my/proposals`	VersionEdit (owner)	Own submissions: collapsed status + competitor count (+ authors on expand, D34); provisional states render as "under review" (D17)
`GET /versioning/my/layer-pending?layerId=…`	VersionEdit (owner)	Own pending proposals as renderable features — feeds the on-demand overlay [A: D34]

Endpoint	Auth (§7)	Purpose
`GET /versioning/features/{layerId}/{featureId}/pending-status`	VersionEdit	On-demand check for one feature: pending-proposal **count + authors** (own and others') [A: D34, amends D18]
`GET /versioning/review/queue?layerId=…`	VersionReview	Queue rows: age, stale/orphan badges, competition sizes, undismissed overlap suggestions
`GET /versioning/review/competitions/{id}`	VersionReview	Compare payload: all proposals (content + author names, D31) + current live state. **Diffs are computed client-side** (reuse `FeatureChangesViewModel` mechanics); server ships raw states
`POST /versioning/review/competitions/{id}/select`	VersionReview	Body: winnerProposalId, per-loser reasons, staleOverride flag (D8)
`POST /versioning/review/competitions/{id}/close-no-winner`	VersionReview	Reasons per proposal (D20)
`POST /versioning/review/group`	VersionReview	Manual grouping: proposalIds[] → one competition (D16)
`POST /versioning/review/suggestions/{id}/dismiss`	VersionReview	D30
`GET /versioning/approval/queue`	VersionApprove	Resolved competitions; fresh stale re-check flags
`POST /versioning/approval/commit`	VersionApprove	Body: competitionIds[], staleOverrides. One CommitBatch, all-or-nothing (E9)
`POST /versioning/approval/competitions/{id}/return`	VersionApprove	Reason (D17)
`GET /versioning/inbox?unreadOnly=`, `POST /versioning/inbox/{id}/read`	any versioning role	Digest notifications (D15/D24)
`GET /versioning/history/{layerId}/{featureId}` (+ `/as-of?at=`)	VersionHistoryRead	Timeline / point-in-time (doc 03 §5.4), D21 visibility applied
`GET /versioning/competitions/{id}/record`	participant or VersionReview+	Post-closure full record (D21)

4. Gating the direct Sync path [A: D26]

Server (the authority): one choke point — `UnitOfWork.SyncFeatureEntitiesAsync` already funnels every domain Sync endpoint; it consults the cached `VersionedLayer` registry and rejects writes to enabled layers with error code `VersionedLayerWriteRejected` (§8). No per-controller changes; ~100 endpoints gated at once.

Client (the convenience): at startup/layer-load the client reads `GET /versioning/layers`. For a versioned layer, `SelectedLayer`'s save command routes to a `VersioningWebDataSource` (implements `IEditableVectorDataSource`; its `SaveChangesAsync` builds a session-submission from the in-memory batch instead of a `FeatureSetChangesDto` sync) — the S5 seam reuse: editors keep the exact same edit tools and save button. Non-versioned layers keep `WebApiDataSource` untouched.

A stale client that somehow syncs directly gets the server rejection with a message telling it to submit via versioning — defense in depth, not UX.

---

5. Client state after submission [A→D34, Q37 modified by Hossein]

The map always renders live truth — the last committed state — and versioning data is **strictly separated** from normal feature queries: layer listing, feature fetch, and rendering pipelines are untouched by versioning. Exactly three on-demand entry points exist on the map side:

1. **Own-pending overlay:** `GET /versioning/my/layer-pending` returns the editor's own pending proposals as separate features, rendered in a distinct pending style alongside live truth (never merged into the layer). Blind rules intact — others' content is never returned.
2. **Per-feature pending-status check:** an explicit action on a selected/edited feature (`GET /versioning/features/…/pending-status`) returning the **count + authors** of pending proposals, including other editors' (D34, amending D18's author-hiding). No automatic call on edit-start — on-demand only.
3. **History** (context menu, §3 history endpoints).

New edits always start from live state; the submission response still carries self-supersede notices (D22) and live-overlap advisories (D30).

Consequence to accept: after submitting, an editor's change "disappears" from the main map into the My-Pending panel / on-demand overlay — Stage 5 must make this transition obvious (post-submit summary dialog + a hint to the overlay toggle).

---

6. Sequence flows (component level; semantics in doc 02)

Submit: EditableFeatureLayer/FeatureTable edits (in-memory, D27) → save command → VersioningWebDataSource → `POST /versioning/sessions` → SubmissionService: validate, per-proposal collision lookup → join/create competition, self-supersede, overlap scan (raw SQL, D32) → persist → response (statuses, counts, advisories) → result dialog; the map keeps showing live truth, and the My-Pending panel reflects the new proposals on its next open/refresh [A→D34, D37].

Review: queue GET → reviewer opens competition → compare payload (raw states) → client-side diffs (geometry on map, attributes in table) → select/close POST → ReviewService guard-checks + RowVersion (E4) → decision records, state flips, N4 if no-winner.

Commit: approval queue GET (fresh stale flags) → commit POST → CommitService runs doc 03 §5.2 inside one BargContext transaction → response lists per-competition outcome; on any gate failure nothing committed (E9) → inbox digests written (N2/N3).

---

7. Authorization mapping [A: FR-4.3, doc 02 §6]

Four new values in Saba's flat `Permission` enum, new **"Versioning"** display group (next free Order block per the enum's conventions): `VersionEdit`, `VersionReview`, `VersionApprove`, `VersionHistoryRead`. The API host's existing pattern (one policy per enum value) picks them up automatically; controllers take `[Authorize(Policy = …)]` as in §3. Roles compose them freely (D5/D10); ownership checks (own

session/proposal) are service-level, not policy-level. The user context handed to services carries **id + display name** so every write stamps both [A: D31].

---

8. Cross-cutting

- **Error model:** ProblemDetails with stable codes the client maps to Persian messages: `VersionedLayerWriteRejected`, `StaleBase`, `SchemaMismatch`, `CompetitionAlreadyResolved`, `CompetitionUnderApproval` (D19 queue), `WithdrawNotAllowed`, `DuplicateActiveProposal` (D22 index race), `NotCompetitionParticipant` (D21).
- **Registry caching:** `VersionedLayer` cached in API memory (invalidate on admin change) and client session (refresh on login/layer-load).
- **Refresh model [A→D37, Q40 modified]: no polling at all in v1** — inbox, own statuses, and queues load on open and via explicit Refresh buttons only. A timer or push channel can be added later without any API change.
- **Localization:** all user-facing strings via the existing Jab.Core resx conventions (Persian primary).
- **Startup:** API recomputes schema signatures [A: D33] and logs any layer whose signature changed (pending proposals there get flagged in queues immediately).
- **Audit separation:** the existing hash-chained security `AuditLog` is untouched; versioning's audit is `DecisionRecord` (append-only, D13). Role/permission changes for versioning flow through the existing user-management audit as today.
- **No push:** nothing in v1 depends on server-initiated messages [A: D15]; if SignalR/gRPC streaming arrives later, it augments the manual-refresh model only.

---

9. Stage-4 detail questions (Q37–Q40) — all ANSWERED 2026-08-13

#	Question	Answer
Q37	Post-submit client display model	**Modified by Hossein → D34**: live truth always; versioning strictly separated from normal queries; on-demand own-pending overlay + per-feature pending-status check revealing **count + authors** (amends D18). Follow-ups confirmed: on-demand invocation only; overlay = own proposals as separate features (not a merged preview).
Q38	Dedicated `/versioning/*` controllers?	Yes → D35.
Q39	Gateway in `Ket.WebApiPersistence`?	Yes → D36.
Q40	Refresh cadence	**Modified by Hossein → D37**: no polling at all in v1; manual refresh only; may improve later.

---

10. Handoff to Stage 5 (UI/UX)

Views to specify: session submit flow (+ post-submit summary, D34 transition), "My pending proposals" panel + on-demand overlay toggle + per-feature pending-status check (D34), review queue (badges: age, stale, orphan, size, suggestions), competition compare view (map diff + attribute diff + delete-as-a-side, D12; reuse `FeatureChangesViewModel` / `FeatureChangesView` mechanics), manual grouping

interaction, approval queue + batch commit + return dialog, inbox (manual refresh, D37), history timeline. Editor-facing status labels must collapse provisional states (doc 02 §7 amendment). Pilot layers are decided (D38: transmission lines + substations).

STAGE 5 OF 6

UI/UX Specification

`docs/features/spatial-versioning/05-ui-ux.md`

> **Status:** BASELINED 2026-08-13 — Q41–Q44 confirmed with recommended defaults → D39–D42 **Last updated:** 2026-08-13 **Prerequisites:** docs 01–04 (decisions D1–D38)

WPF views for the versioning workflow. Semantics are fixed by docs 02–04; this document specifies screens, layouts, affordances, and status language. Visual polish (exact colors, spacing) follows the existing Jab.Wpf theme at implementation time.

Markers: **[A]** = follows from a D-decision. Former [P] proposals were all confirmed 2026-08-13 and now read **[A→D39]**…**[A→D42]**.

---

0. Principles

- **Persian-first**, strings via the existing Jab.Core resx conventions; layouts RTL-aware.
- **No timers [A: D37]:** every list view has a prominent Refresh button and a "last refreshed at" label; data loads on open.
- **Strict separation [A: D34]:** the normal map experience is untouched. Versioning enters through: the save routing (invisible), three on-demand map actions (own-pending overlay, pending-status check, history), and the role-gated Versioning menu.
- **Server is authority:** guard functions from `Sta.Versioning` only enable/disable buttons; every action handles a server rejection gracefully (error codes → doc 04 §8).
- **Editor-facing status collapse [A: D17, doc 02 §7 amendment]:** editors never see provisional review outcomes. Internal → shown label mapping:

Internal proposal state	Editor sees
Submitted (singleton)	در انتظار بررسی — "Pending review"
Submitted (in competition)	در رقابت (N) — "In competition (N)"
SelectedForApproval	در حال بررسی — "Under review"
ProvisionallyRejected	در حال بررسی — "Under review" (identical, deliberately)
Committed	تثبیت‌شده — "Committed"
Rejected	رد شده — "Rejected" (+ reason)
Withdrawn	پس‌گرفته — "Withdrawn" (+ cause)

1. Navigation & map entry points

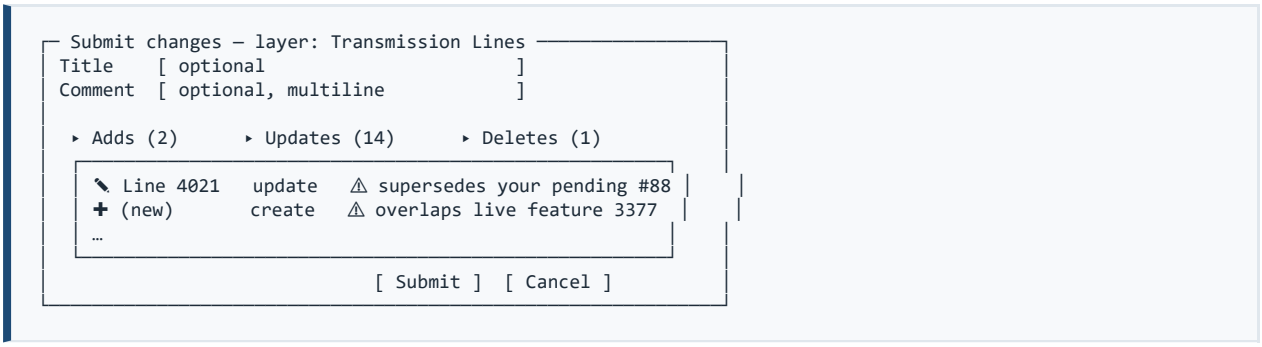
Saba shell menu "Versioning" (items appear per permission, doc 04 §7): My Pending (VersionEdit) · Review Queue (VersionReview) · Approval Queue (VersionApprove) · Inbox (any role, unread badge updates on refresh only [A: D37]).

Map/feature context (versioned layers only, from the D26 registry):

- *Show my pending changes* — toggles the own-pending overlay [A: D34]: proposals drawn as separate features in the pending style; delete-proposals drawn as a hatched ghost over the live feature.
- *Check pending status* — on the selected feature; result popup: "N pending proposals: ‹author names›; one of them is yours" [A: D34]. No content shown.
- *History…* — opens the feature timeline (§8), permission VersionHistoryRead.

2. Submit flow

Editing and digitizing are exactly today's tools [A: D27]; the Save command on a versioned layer opens the **Submit dialog** instead of syncing:

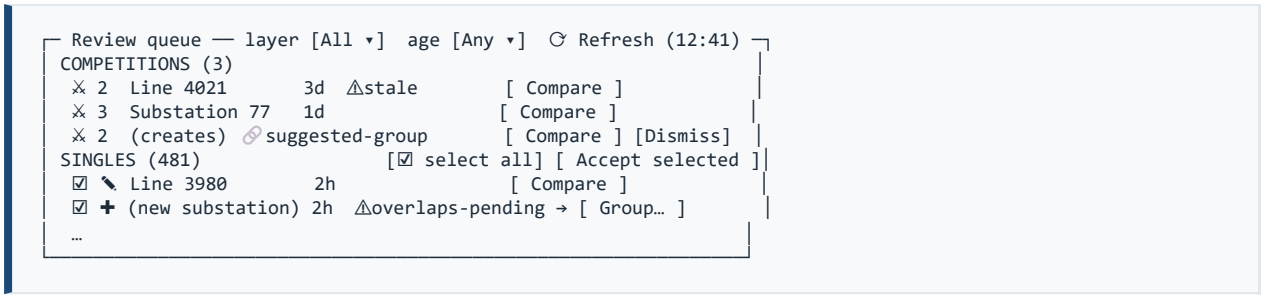


Self-supersede warnings (D22) and live-overlap advisories (D30, kind 2) appear inline per row — advisories are informational, never blocking. After submission, the **Result dialog** lists per-proposal outcomes (pending / in competition (N) / superseded #id), then the map simply continues showing live truth [A: D34]; a status-bar hint points to the overlay toggle and My Pending. If the submission was rejected wholesale (`VersionedLayerWriteRejected` race, validation), the draft stays intact locally.

3. My Pending panel (dockable)

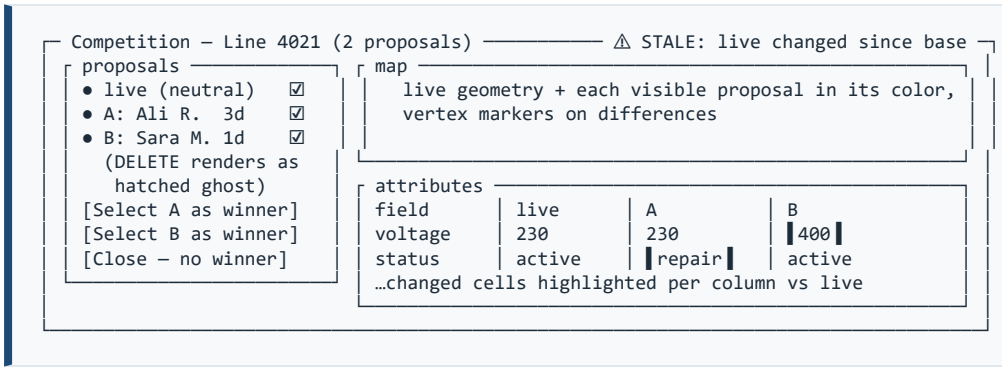
Tree grouped by session (title · submitted at · comment) → proposal rows: feature label, change-type icon, collapsed status (§0 table), competition count with expandable author list [A: D34]. Toolbar: Refresh · Withdraw proposal (guard D25, confirm) · Withdraw session (guard D9, confirm) · Zoom to feature · Show on map (overlay). A closed competition row links to its full record (D21). Rejected rows show the reason inline.

4. Review Queue (VersionReview)



- Badges: age, stale (D8), orphan (D23 — opens in reject-only mode), schema-mismatch (doc 03 §5.3), overlap suggestion (D30).
- **Bulk accept** of selected singles = one confirmation dialog with count + expandable list [A→D40]; per-row failures (E4 races, guards) reported in the result, the rest succeed.
- **Group…** merges selected create-proposals of the same layer into one competition (D16); suggestion chips offer one-click grouping of the suggested pair; Dismiss records the reviewer + name (D30/D31).

5. Competition Compare view (the centerpiece)



- **N-way attribute grid** [A→D39]: one column per proposal beside live; changed cells highlighted (reuse `DictionaryHelper.GetChangedAttributes` per column). Session comment and author (D31) in each column header tooltip.
- Delete proposals appear as an explicit DELETE side (D12): hatched ghost on map, struck column in the grid.
- **Select winner** → dialog: rejection reason per loser (or one applied to all) + stale-override checkbox when the stale banner is up (D8) → confirm. **Close — no winner** → reasons dialog (D20, immediate finality warning). Orphaned competitions (D23) show only reject actions.
- Banner area: stale (base vs live info), orphan, schema-mismatch (lists problem fields, doc 03 §5.3).

6. Approval Queue (VersionApprove)

Rows: feature/layer · winner author · resolving reviewer · resolved at · fresh-stale badge (re-checked on refresh). Multi-select → **Commit** (one confirmation with count; all-or-nothing result per E9 — failures listed with codes, nothing partial). Row detail = Compare view read-only + decision summary → **Return**

with reason (D17). Same-person review+approve is permitted; the UI shows both identities on the record (D10).

7. Inbox

Manual refresh only [A: D37]. Digest rows (D24): type icon · title ("Session 'X': 480 committed, 15 rejected") · expandable per-feature detail with reasons · read/unread. Row actions: jump to session (My Pending) or competition record (D21 rules).

8. History timeline (per feature)

Opened from the feature context menu. Newest-first rows: commit timestamp · editor name · approver name (D31) · session comment · competition-size chip; topmost row = current live. Actions: **Show this version on map** — historical geometry drawn alongside current in a distinct style [A→D41]; **Compare two versions** — 2-way reuse of the compare view; **As-of…** date picker (doc 03 §5.4). Competition-size chip links to the closed record (access per D21). A committed delete appears as the terminal row.

9. Color & badge semantics (theme tokens at implementation)

Meaning	Convention
Pending (own overlay, queue rows)	orange family
In competition	orange + count badge ⚔
Committed / accepted	green
Rejected / delete-side	red; deletes hatched
Stale	amber ⚠
Orphaned	gray ⚠
Historical geometry	desaturated blue, dashed outline
Proposal colors in compare	fixed distinct palette (A, B, C…), colorblind-safe

10. Failure & empty states

Every server rejection maps an error code (doc 04 §8) to a Persian message with the next step ("این عارضه در حال تأیید است — پس از تثبیت دوباره ارسال کنید" for `CompetitionUnderApproval`, etc.). Empty states say what the view is *for* ("No pending proposals — changes you submit on versioned layers appear here"). Destructive actions (withdraw, reject-all, commit, return) always confirm and always echo the count.

11. Stage-5 detail questions (Q41–Q44) — all ANSWERED 2026-08-13

Confirmed with the recommended defaults → **D39–D42** (doc 01 §2). Table kept for rationale/impact notes.

#	Question	Recommendation	Impact if changed
Q41	Compare view uses a single **N-way attribute grid** (live + one column per proposal), not pairwise tabs?	Yes (§5) — N is almost always 2–3	Pairwise tabs scale to large N but hide the overview; N-way grid is the reviewer's whole job on one screen
Q42	Bulk-accepting singles takes **one confirmation** (count + expandable list), no per-item step?	Yes (§4) — R5 throughput	Per-item ceremony recreates the 500-click problem the fast path exists to solve
Q43	History "show version on map" **overlays** the historical geometry beside current (distinct style), rather than temporarily replacing the live rendering?	Yes (§8) — the map never lies (D34 spirit)	Replace-mode previews are clearer for big changes but show non-live data as the layer
Q44	Versioning views live **inside** `IRI.Maptor.Jab.Wpf` (own folder, beside `FeatureChangesView`) rather than a new UI project?	Yes — lowest friction, matches precedent	A separate `Jab.Wpf.Versioning` project isolates dependencies but adds packaging overhead for one consumer (v1)

12. Handoff to Stage 6 (implementation plan)

All design inputs are decided once Q41–Q44 are confirmed. Stage 6 turns docs 02–05 into milestones over the pilot (D38: transmission lines + substations); suggested build order to elaborate there: shared model + persistence → submission path + Sync gate → review (queue, compare, decisions) → commit + history → notifications/inbox → map entry points (overlay, status check, timeline) → pilot enablement + operator walkthrough.

STAGE 6 OF 6

Implementation Plan

`docs/features/spatial-versioning/06-implementation-plan.md`

> **Status:** BASELINED 2026-08-13 — Q45–Q46 answered → D43–D44; **M1 started 2026-08-13 Last updated:** 2026-08-13 **Prerequisites:** docs 01–05, all baselined (decisions D1–D42)

This plan turns the design into six milestones over the pilot (D38: **transmission lines

- substations**). Sizes are relative (S/M/L) — calendar estimates depend on availability and are deliberately not faked here. Each milestone ends demoable; the doc 02 §9 walkthroughs (A–F) double as the acceptance-test scripts.

---

1. Ground rules

- Build strictly in milestone order — each depends on the previous.
- Nothing outside the pilot layers is enabled in production until M6 is evaluated.
- Every deviation from docs 01–05 discovered during implementation goes back into the decision log (doc 01 §2) before the code merges — the docs stay the truth.
- Deferred list (doc 01 §5) is off-limits: no merge features, no assignments, no push, no persisted undo, no topology enforcement.

---

2. Milestones

M1 — Foundation (size M)

Scope: create `IRI.Maptor.Sta.Versioning` (entities, enums, guard functions, DTOs, serialization contract + schema-signature computation) and `IRI.Maptor.Ket.VersioningPersistence` (EF configurations, `AddMaptorVersioning()`); EF migration creating the `versioning` schema — 9 tables incl. the D19/D22 filtered unique indexes and the Competition ↔ Proposal circular FK (no cascade); `VersionedLayer` registry + startup signature computation (D33); minimal registry admin endpoint. **Spike (do first):** verify the custom provider's `geometry` pass-through mapping and a spatial index on a `versioning`-schema table (D32) against a copy of the Saba DB. **Accept:** migration applies cleanly to a Saba DB copy; signatures computed for the two pilot layers; guard-function unit tests green.

M2 — Submission + Sync gate (size L)

Scope: SubmissionService (validation, collision → join/create competition, self-supersede D22, overlap scan D30/D32, N1 notifications); Sync gate at the `UnitOfWork.SyncFeatureEntitiesAsync` choke point (D26); endpoints: sessions, my/*, layers, pending-status, layer-pending (doc 04 §3); client: `VersioningWebClient` + `VersioningWebDataSource` in Ket.WebApiPersistence (D36), save routing, Submit + Result dialogs (doc 05 §2); the four `Version*` permissions + policies (doc 04 §7). **Accept:**

walkthrough **B** (bulk 500-feature session → singletons) passes end-to-end on a pilot layer; direct Sync to a versioned layer is rejected with `VersionedLayerWriteRejected`; a second editor's colliding submission joins the competition and both owners get N1.

M3 — Review (size L)

Scope: review-queue endpoint + view (badges, bulk accept D40, manual grouping D16, suggestion dismiss); compare-payload endpoint; Compare view (N-way grid D39, map styles, delete-as-a-side D12); select-winner / close-no-winner with reasons + stale override (D8/D20); decision records; E4 concurrency handling. **Accept:** walkthroughs **A** (competition), **E** (delete vs edit), **F** (manual grouping of creates) pass; a 500-singleton queue is bulk-accepted in one confirmation; two concurrent reviewers on one competition → second gets `CompetitionAlreadyResolved`.

M4 — Commit + history (size L)

Scope: CommitService (doc 03 §5.2: stale + schema gates, copy-on-write, author id + display-name stamping D31, all-or-nothing batch E9); approval queue + view; return flow (D17 reopen); FeatureHistory; history timeline view + as-of (D41 overlay); post-closure record view (D21). **Accept:** walkthroughs **C** (stale override at review and again at approval) and **D** (approver return → re-resolution) pass; timeline reconstructs a feature across 3 commits incl. a delete; a batch with one schema-blocked competition commits nothing.

M5 — Editor experience (size M)

Scope: My Pending panel (status-collapse table, doc 05 §0 — verify provisional states are indistinguishable); inbox with digests (D24, manual refresh D37); map entry points: own-pending overlay, pending-status check (D34), history context-menu entry; N2–N6 notification writing; error-code → Persian message mapping. **Accept:** an editor whose proposal is provisionally rejected sees only "under review" until commit; pending-status check shows count + authors but never content; all doc 04 §8 error codes render localized messages.

M6 — Pilot enablement (size S–M)

Scope: enable registry rows for transmission lines + substations; role setup for the pilot cohort; operator walkthrough script (Persian); monitoring queries (queue depth, oldest pending age, stale count); feedback capture sheet. **Accept:** a real session by ≥2 editors + 1 reviewer/approver completes the full cycle on staging (Q46), then on production pilot; evaluation review scheduled (~2–4 weeks of pilot use) before widening layers.

---

3. Test strategy

- **Unit** (Sta.Versioning): guard functions, serialization canonical form, signature computation, tolerant-mapping rules (doc 03 §5.3 cases).
- **Integration** (SQL Server, DB copy): commit transaction (gates, copy-on-write, rollback), filtered-index race behavior (D19/D22), E4 optimistic concurrency, Sync gate, overlap scan correctness.
- **Acceptance:** walkthroughs A–F as scripted scenarios, distributed across M2–M4 as listed above.
- **UI:** manual test scripts per view; stub-data window previews where the existing render-harness approach applies (verify current state of that harness before relying on it).

4. Risk register (implementation-phase)

Risk	Mitigation
Geometry pass-through or spatial index misbehaves on `versioning`-schema tables	M1 spike, before anything is built on it
Circular FK (Competition.Winner ↔ Proposal.Competition) trips EF migrations	Configure winner FK without cascade, add in a follow-up migration step if needed
R1 schema drift during the pilot	D33 startup detection + doc 03 §5.3 gate are in from M1; add a pending-proposal compatibility check to the deploy checklist
Reviewer throughput worse than assumed (R5)	M3 acceptance includes the 500-singleton bulk case with a stopwatch; revisit UI before M6 if painful
Duplicate creates slip past manual grouping (D16 residual risk)	Pilot feedback sheet asks reviewers explicitly; count grouped-vs-missed in monitoring
Pilot users bypass versioning out of habit	D26 server gate makes bypass impossible, not just discouraged; walkthrough script explains the new save behavior

5. Rollout & rollback

Dev → **the current working system, which serves as staging** (D44 — data loss there is acceptable) → pilot (2 layers, small cohort) → evaluation → widen layer-by-layer (S6 opt-in was built for exactly this). Rollback for a layer = flip `IsVersioningEnabled` off, returning it to direct Sync; disabling is **blocked while open proposals exist** (D43) — the admin resolves or withdraws them first, so history never has a hole.

6. Definition of done (planning → implementation)

Docs 01–06 baselined; D1–D42 decided; Q45–Q46 are the only open items and are operational, not design. Implementation starts at M1 on Hossein's go — no code is written as part of the planning effort.

7. Stage-6 operational questions (Q45–Q46) — ANSWERED 2026-08-13

#	Question	Answer
Q45	Disabling versioning with open proposals?	**Blocked** until resolved/withdrawn → D43.
Q46	Test environment?	**The current working system is the staging** (data loss acceptable there) → D44. M1 migration test and M6 dry run happen on it directly.